

# Docker Cheat Sheet

36 tables · 268 concepts · Updated 2026-08-01 · [cheatgrid.com](#)

Docker is a containerization platform that packages applications with their dependencies into lightweight, portable containers. It revolutionized software deployment by solving the "works on my machine" problem through consistent, isolated environments that run identically across development, testing, and production. Docker containers share the host OS kernel, making them far more efficient than traditional virtual machines while maintaining strong isolation. Docker Engine 29 (2026) ships BuildKit v0.32 as its default builder and Docker Compose V5 as the standard for multi-container orchestration; the legacy `docker-compose` v1 binary was fully removed in April 2025, so only `docker compose` (no hyphen) works now. The 2025-2026 release cycle significantly expanded Docker's scope: **Docker Model Runner** enables local AI inference with an OpenAI-compatible API; **Docker Scout** provides integrated vulnerability scanning and SBOM analysis; **Docker Hardened Images (DHI)** offer free, distroless-based images with CVE patches within 24 hours; **Docker Build Cloud** offloads builds to remote infrastructure for faster CI, while the broader **Docker Offload** (generally available since April 2026) runs entire containers, not just builds, on a remote Docker daemon for Business subscribers; **Gordon** ( `docker ai` ) is an AI-powered assistant that manages containers, debugs failures, and rewrites Dockerfiles from the CLI; and the **Docker MCP Toolkit** ( `docker mcp` ) makes it trivial to run and connect 120+ containerized Model Context Protocol servers to any AI client. The key mental model: **a container is a running instance of an image**, where images are immutable blueprints built in layers, and layers are cached for speed — understanding this layer caching mechanism is critical for optimizing build times and image sizes.

## CONTENTS

|                                                |    |
|------------------------------------------------|----|
| Container Lifecycle Management . . . . .       | 2  |
| Container Inspection and Interaction . . . . . | 3  |
| Image Management . . . . .                     | 3  |
| Dockerfile Instructions . . . . .              | 4  |
| Build Optimization Techniques . . . . .        | 5  |
| Port Publishing and Networking . . . . .       | 6  |
| Network Modes . . . . .                        | 6  |
| Network Management . . . . .                   | 7  |
| Volume Management . . . . .                    | 8  |
| Volume Commands . . . . .                      | 8  |
| Resource Constraints . . . . .                 | 9  |
| Restart Policies . . . . .                     | 9  |
| Health Checks . . . . .                        | 10 |
| Docker Compose Services . . . . .              | 10 |
| Docker Compose Commands . . . . .              | 11 |
| Docker Compose Advanced Features . . . . .     | 12 |
| Registry and Authentication . . . . .          | 13 |
| Image Tagging Strategies . . . . .             | 13 |
| Cleanup and Maintenance . . . . .              | 14 |
| Logging Drivers . . . . .                      | 14 |
| Security Practices . . . . .                   | 15 |
| Signal Handling . . . . .                      | 16 |
| Context and Remote Docker . . . . .            | 16 |
| Docker Swarm Basics . . . . .                  | 17 |
| Environment Variables . . . . .                | 17 |

|                                       |    |
|---------------------------------------|----|
| Inspect and Debug                     | 18 |
| Copy Operations                       | 18 |
| BuildKit Advanced Features            | 19 |
| Docker Scout Commands                 | 19 |
| Docker Model Runner                   | 20 |
| Docker Build Cloud and Docker Offload | 20 |
| Compose Dependency Conditions         | 21 |
| Distroless and Minimal Images         | 22 |
| Docker AI (Gordon)                    | 22 |
| Docker MCP Toolkit                    | 23 |
| Docker DHI CLI                        | 23 |
| References                            | 24 |

TABLE 1

## Container Lifecycle Management

These commands move a container through every phase of its life — from initial creation, through running and pausing, to graceful or forced termination. Knowing which command uses which signal (and which preserves the writable layer) is what separates clean production shutdowns from corrupted state.

| Command               | Example                                                | Description                                                                                                                                                                                                                                          |
|-----------------------|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker run</b>     | <code>docker run -d -p 8080:80 --name web nginx</code> | <ul style="list-style-type: none"> <li>Creates and starts a <b>new container</b> from an image</li> <li><code>-d</code> runs detached, <code>-p</code> maps ports, <code>--name</code> assigns a custom name. Most commonly used command.</li> </ul> |
| <b>docker start</b>   | <code>docker start web</code>                          | <ul style="list-style-type: none"> <li>Starts an <b>existing stopped</b> container</li> <li>preserves all container state and configuration.</li> </ul>                                                                                              |
| <b>docker stop</b>    | <code>docker stop web</code>                           | Gracefully stops a running container by sending <b>SIGTERM</b> then SIGKILL after grace period (default 10s).                                                                                                                                        |
| <b>docker restart</b> | <code>docker restart web</code>                        | <ul style="list-style-type: none"> <li><b>Stops then starts</b> the same container instance</li> <li>equivalent to <code>stop + start</code> ; preserves the writable layer.</li> </ul>                                                              |
| <b>docker pause</b>   | <code>docker pause web</code>                          | <ul style="list-style-type: none"> <li><b>Freezes</b> all processes in a container using the cgroups freezer</li> <li>no CPU usage, but memory stays allocated; instant resume with <code>unpause</code> .</li> </ul>                                |
| <b>docker unpause</b> | <code>docker unpause web</code>                        | <ul style="list-style-type: none"> <li><b>Resumes a paused container</b></li> <li>restores execution exactly where it stopped.</li> </ul>                                                                                                            |
| <b>docker rm</b>      | <code>docker rm -f web</code>                          | <ul style="list-style-type: none"> <li><b>Removes a stopped container</b></li> <li><code>-f</code> forces removal of running containers by SIGKILL-ing them first (no grace period).</li> </ul>                                                      |
| <b>docker kill</b>    | <code>docker kill web</code>                           | <ul style="list-style-type: none"> <li>Immediately stops a container by sending <b>SIGKILL</b></li> <li>no graceful shutdown, use for unresponsive containers.</li> </ul>                                                                            |
| <b>docker create</b>  | <code>docker create --name web nginx</code>            | <ul style="list-style-type: none"> <li>Creates a container <b>without starting it</b></li> <li>useful for preparing containers before <code>start</code> .</li> </ul>                                                                                |
| <b>docker wait</b>    | <code>docker wait web</code>                           | <ul style="list-style-type: none"> <li>Blocks until a container stops, then prints its <b>exit code</b></li> <li>useful in scripts.</li> </ul>                                                                                                       |
| <b>docker rename</b>  | <code>docker rename web web-old</code>                 | <ul style="list-style-type: none"> <li><b>Renames</b> an existing container</li> <li>works on running and stopped containers; log driver tags keep the original name.</li> </ul>                                                                     |
| <b>docker update</b>  | <code>docker update --cpus 2 --memory 1g web</code>    | <ul style="list-style-type: none"> <li>Dynamically <b>updates resource limits</b> and restart policy on running containers</li> <li>cannot change image, env vars, ports, or volumes.</li> </ul>                                                     |

TABLE 2

## Container Inspection and Interaction

Once a container is running, day-to-day work is about seeing what it is doing and reaching inside it. These commands cover listing containers, reading their logs and metrics, running ad-hoc commands inside them, and pulling out the metadata Docker keeps about each one.

| Command               | Example                                                                   | Description                                                                                                                                                                                                                                                                                                   |
|-----------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker ps</b>      | <code>docker ps -a</code>                                                 | <ul style="list-style-type: none"> <li>Lists containers</li> <li>default shows <b>running only</b>, <code>-a</code> includes stopped; <code>--filter health=unhealthy</code> filters by health state; <code>--format '{{.HealthStatus}}'</code> prints health as a dedicated field (Engine 29.5+).</li> </ul> |
| <b>docker logs</b>    | <code>docker logs -f --tail 100 web</code>                                | <ul style="list-style-type: none"> <li>Shows <b>container stdout/stderr</b> of the main process</li> <li><code>-f</code> follows live output, <code>--tail</code> limits lines.</li> </ul>                                                                                                                    |
| <b>docker exec</b>    | <code>docker exec -it web bash</code>                                     | <ul style="list-style-type: none"> <li>Runs a <b>new process</b> inside a running container</li> <li><code>-it</code> allocates interactive TTY for shell access.</li> </ul>                                                                                                                                  |
| <b>docker attach</b>  | <code>docker attach web</code>                                            | <ul style="list-style-type: none"> <li>Connects to container's <b>main process (PID 1)</b> stdin/stdout/stderr</li> <li>Ctrl+C sends SIGINT and usually stops the container; detach with Ctrl+P, Ctrl+Q.</li> </ul>                                                                                           |
| <b>docker inspect</b> | <code>docker inspect --format '{{.NetworkSettings.IPAddress}}' web</code> | <ul style="list-style-type: none"> <li>Returns <b>detailed JSON</b> with all container metadata (config, state, network, mounts)</li> <li><code>--format</code> Go templates extract single fields.</li> </ul>                                                                                                |
| <b>docker stats</b>   | <code>docker stats --no-stream web</code>                                 | <ul style="list-style-type: none"> <li>Live stream of <b>CPU, memory, network, disk I/O</b> metrics</li> <li><code>--no-stream</code> prints one snapshot then exits (use in scripts).</li> </ul>                                                                                                             |
| <b>docker top</b>     | <code>docker top web</code>                                               | <ul style="list-style-type: none"> <li>Shows <b>running processes</b> inside a container, using the host's <code>ps</code></li> <li>PID column reports <b>host PIDs</b>, not the container's PID 1.</li> </ul>                                                                                                |
| <b>docker port</b>    | <code>docker port web</code>                                              | <ul style="list-style-type: none"> <li>Lists all <b>published port mappings</b> for a container</li> <li>shows host port → container port bindings (only ports published with <code>-p</code> / <code>-P</code>).</li> </ul>                                                                                  |
| <b>docker diff</b>    | <code>docker diff web</code>                                              | Shows <b>filesystem changes</b> vs the starting image (A=added, C=changed, D=deleted).                                                                                                                                                                                                                        |

TABLE 3

## Image Management

Commands for getting images onto and off your machine: building from a Dockerfile, pulling and pushing through registries, tagging, inspecting layers, and moving images between hosts as tar archives. The two biggest gotchas live here too — `save` vs `export` (image vs container, layered vs flat) and `docker commit` as a production anti-pattern.

| Command               | Example                                        | Description                                                                                                                                                                                      |
|-----------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker build</b>   | <code>docker build -t myapp:1.0 .</code>       | <ul style="list-style-type: none"> <li>Builds an image from a Dockerfile</li> <li><code>-t</code> tags it; <code>.</code> is the <b>build context</b> directory sent to the daemon.</li> </ul>   |
| <b>docker images</b>  | <code>docker images</code>                     | Lists <b>local images</b> with repository, tag, image ID, creation date, and size.                                                                                                               |
| <b>docker pull</b>    | <code>docker pull nginx:alpine</code>          | <ul style="list-style-type: none"> <li>Downloads an image from a <b>registry</b> (default Docker Hub)</li> <li>only missing layers are fetched thanks to content-addressable storage.</li> </ul> |
| <b>docker push</b>    | <code>docker push myuser/myapp:1.0</code>      | <ul style="list-style-type: none"> <li>Uploads an image to a <b>registry</b></li> <li>requires <code>docker login</code> first.</li> </ul>                                                       |
| <b>docker tag</b>     | <code>docker tag myapp:1.0 myapp:latest</code> | <ul style="list-style-type: none"> <li>Adds a <b>new name</b> pointing to the same image ID</li> <li>does not copy or duplicate layers.</li> </ul>                                               |
| <b>docker rmi</b>     | <code>docker rmi nginx:alpine</code>           | <ul style="list-style-type: none"> <li><b>Removes an image</b></li> <li>fails if any container (running or stopped) references it unless <code>-f</code> forces removal.</li> </ul>              |
| <b>docker history</b> | <code>docker history nginx</code>              | <ul style="list-style-type: none"> <li>Shows each <b>layer</b> in an image with its size and creating command</li> <li>the go-to tool for diagnosing image bloat.</li> </ul>                     |

| Command              | Example                                         | Description                                                                                                                                                                                                              |
|----------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker save</b>   | <code>docker save -o app.tar myapp:1.0</code>   | Exports an <b>image</b> to a tar archive, preserving all layers and metadata.                                                                                                                                            |
| <b>docker load</b>   | <code>docker load -i app.tar</code>             | <ul style="list-style-type: none"> <li>Imports an image from a <code>docker save</code> tarball</li> <li>restores all layers and original tags.</li> </ul>                                                               |
| <b>docker import</b> | <code>docker import rootfs.tar myapp:1.0</code> | <ul style="list-style-type: none"> <li>Creates an image from a raw <b>filesystem tarball</b></li> <li>result is a single layer with no build history.</li> </ul>                                                         |
| <b>docker export</b> | <code>docker export web -o web.tar</code>       | <ul style="list-style-type: none"> <li>Exports a <b>container's</b> filesystem as a flat tar</li> <li>layers collapsed, image metadata lost.</li> </ul>                                                                  |
| <b>docker commit</b> | <code>docker commit web myapp:snapshot</code>   | <ul style="list-style-type: none"> <li>Creates a new image from a container's filesystem changes</li> <li>fine for ad-hoc debugging, an <b>anti-pattern</b> for production (no Dockerfile, not reproducible).</li> </ul> |
| <b>docker search</b> | <code>docker search --limit 5 nginx</code>      | <ul style="list-style-type: none"> <li>Searches <b>Docker Hub</b> only (not arbitrary registries)</li> <li><code>--filter</code> narrows by stars or official status.</li> </ul>                                         |

TABLE 4

## Dockerfile Instructions

A Dockerfile is a recipe of instructions that BuildKit executes top-to-bottom to assemble an image, with most instructions producing a new layer. The instructions below are the building blocks every Dockerfile uses, and the trickiest part is knowing which ones run at build time versus runtime, and which ones only attach metadata.

| Instruction       | Example                                                                                                                                                                                                                                                                                                            | Description                                                                                                                                                                                                                                                                                                  |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FROM</b>       | <code>FROM node:18-alpine</code>                                                                                                                                                                                                                                                                                   | <ul style="list-style-type: none"> <li>Sets the <b>base image</b> and starts a build stage</li> <li>every Dockerfile begins with FROM (after optional ARG/parser directives)</li> <li>pin a specific tag or digest, never <code>latest</code> in production.</li> </ul>                                      |
| <b>RUN</b>        | <code>RUN apt-get update &amp;&amp; \</code><br><code>&lt;br&gt; apt-get install -y curl  </code><br>• Executes commands during <b>image build</b> and commits the result as a new layer<br>• always chain <code>apt-get update</code> and <code>apt-get install</code> in the same RUN to avoid stale-cache bugs. |                                                                                                                                                                                                                                                                                                              |
| <b>CMD</b>        | <code>CMD ["npm", "start"]</code>                                                                                                                                                                                                                                                                                  | <ul style="list-style-type: none"> <li>Provides the <b>default command</b> (or default args for ENTRYPOINT) when a container starts</li> <li>replaced by any command passed to <code>docker run</code></li> <li>prefer the exec form (JSON array) for correct signal handling.</li> </ul>                    |
| <b>ENTRYPOINT</b> | <code>ENTRYPOINT ["python", "app.py"]</code>                                                                                                                                                                                                                                                                       | <ul style="list-style-type: none"> <li>Configures the container to run as a <b>fixed executable</b></li> <li>args to <code>docker run</code> are appended to it; override only with <code>--entrypoint</code></li> <li>when both ENTRYPOINT and CMD are exec form, CMD becomes default arguments.</li> </ul> |
| <b>COPY</b>       | <code>COPY package*.json ./</code>                                                                                                                                                                                                                                                                                 | <ul style="list-style-type: none"> <li>Copies files and directories from the <b>build context</b> into the image</li> <li>preferred default for local files — predictable, no hidden behavior.</li> </ul>                                                                                                    |
| <b>ADD</b>        | <code>ADD app.tar.gz /app/</code>                                                                                                                                                                                                                                                                                  | <ul style="list-style-type: none"> <li>Like COPY, but also <b>auto-extracts local tar archives</b> and can fetch remote URLs</li> <li>use COPY unless you specifically need extraction or URL/Git fetching.</li> </ul>                                                                                       |
| <b>WORKDIR</b>    | <code>WORKDIR /app</code>                                                                                                                                                                                                                                                                                          | <ul style="list-style-type: none"> <li>Sets the <b>working directory</b> for the RUN, CMD, ENTRYPOINT, COPY, and ADD that follow</li> <li>creates the directory if missing — prefer absolute paths.</li> </ul>                                                                                               |
| <b>ENV</b>        | <code>ENV NODE_ENV=production</code>                                                                                                                                                                                                                                                                               | <ul style="list-style-type: none"> <li>Sets <b>environment variables</b> that persist in subsequent build steps and in running containers</li> <li>never use for secrets — values leak into image layers and <code>docker history</code>.</li> </ul>                                                         |

| Instruction        | Example                                             | Description                                                                                                                                                                                                                                                                       |
|--------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ARG</b>         | ARG VERSION=1.0                                     | <ul style="list-style-type: none"> <li>Defines a <b>build-time variable</b> settable with <code>--build-arg</code></li> <li>not present in the running container's environment, and re-scoped per stage in multi-stage builds.</li> </ul>                                         |
| <b>EXPOSE</b>      | EXPOSE 8080                                         | <ul style="list-style-type: none"> <li><b>Documents</b> which ports the container listens on (image metadata only)</li> <li>does NOT publish ports — <code>docker run -p</code> or <code>-P</code> is still required for host access.</li> </ul>                                  |
| <b>VOLUME</b>      | VOLUME /data                                        | <ul style="list-style-type: none"> <li>Declares a path as a <b>mount point</b> for an external/anonymous volume</li> <li>with the legacy builder, writes to that path in later RUN steps are discarded (kept under BuildKit).</li> </ul>                                          |
| <b>USER</b>        | USER node                                           | <ul style="list-style-type: none"> <li>Sets the <b>UID/GID</b> used by subsequent RUN, CMD, and ENTRYPOINT and at runtime</li> <li>place it after any RUN that needs root (package install, chown) — running as non-root is a security baseline.</li> </ul>                       |
| <b>LABEL</b>       | LABEL version="1.0"                                 | <ul style="list-style-type: none"> <li>Attaches <b>metadata</b> to the image as key=value pairs</li> <li>visible via <code>docker inspect</code> and filterable with <code>docker image ls --filter</code>, no runtime behavior.</li> </ul>                                       |
| <b>HEALTHCHECK</b> | HEALTHCHECK CMD curl -f http://localhost/    exit 1 | <ul style="list-style-type: none"> <li>Defines a command Docker runs periodically to test if the container is <b>healthy</b> (exit 0=healthy, 1=unhealthy, 2 reserved)</li> <li>only flips the container's health status; Compose/Swarm/K8s decide whether to restart.</li> </ul> |
| <b>STOPSIGNAL</b>  | STOPSIGNAL SIGQUIT                                  | <ul style="list-style-type: none"> <li>Sets the <b>signal sent to PID 1</b> when stopping the container</li> <li>defaults to SIGTERM; signals only reach the process directly when CMD/ENTRYPOINT use exec form.</li> </ul>                                                       |
| <b>SHELL</b>       | SHELL ["/bin/bash", "-c"]                           | <ul style="list-style-type: none"> <li>Overrides the <b>shell used for shell-form</b> RUN/CMD/ENTRYPOINT</li> <li>default is <code>["/bin/sh", "-c"]</code> on Linux and <code>["cmd", "/S", "/C"]</code> on Windows — essential for picking PowerShell on Windows.</li> </ul>    |
| <b>ONBUILD</b>     | ONBUILD COPY . /app                                 | <ul style="list-style-type: none"> <li>Registers a <b>trigger instruction</b> that runs only when this image is used as a <b>FROM</b> base in a downstream build</li> <li>useful for framework base images; triggers do not cascade to grandchildren.</li> </ul>                  |

TABLE 5

## Build Optimization Techniques

These techniques shrink final images, speed up rebuilds, and keep secrets out of layer history. Most are built on BuildKit, the modern Dockerfile frontend ( `# syntax=docker/dockerfile:1` ) — multi-stage stages, cache mounts, and secret mounts only behave as described when BuildKit is the active builder.

| Technique                 | Example                                                             | Description                                                                                                                                                                                                                                                                                                                                       |
|---------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Multi-stage builds</b> | FROM golang AS builder<br>FROM alpine<br>COPY --from=builder /app . | <ul style="list-style-type: none"> <li>Uses multiple <b>FROM</b> statements to create <b>separate build stages</b>; only artifacts copied into the final stage end up in the published image</li> <li>intermediate stages (compilers, dev tools) are discarded entirely, drastically reducing size and attack surface.</li> </ul>                 |
| <b>Layer caching</b>      | COPY package*.json ./<br>RUN npm install<br>COPY . .                | <ul style="list-style-type: none"> <li>Docker caches each instruction's output as a layer; if a layer's inputs change, that layer <b>and every layer after it</b> must rebuild</li> <li>order instructions least- to most-frequently changed (manifests + install before <code>COPY . .</code> ) to maximize hits.</li> </ul>                     |
| <b>COPY --link</b>        | COPY --link requirements.txt .                                      | <ul style="list-style-type: none"> <li>Creates an <b>independent layer</b> that doesn't depend on earlier ones — changes to instructions above it don't invalidate it</li> <li>enables remote cache reuse ( <code>--cache-from</code> ) and image rebasing across multi-stage and large builds.</li> </ul>                                        |
| <b>.dockerignore</b>      | node_modules<br>*.log<br>.git                                       | <ul style="list-style-type: none"> <li>Excludes files from the <b>build context</b> (the bundle sent to the BuildKit daemon), not from the image directly</li> <li>shrinks context upload, blocks accidental <code>COPY</code> of <code>node_modules</code> / <code>.git</code> / secrets, and keeps the cache stable across machines.</li> </ul> |

| Technique                        | Example                                                                                                               | Description                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>BuildKit cache mounts</b>     | <code>RUN --mount=type=cache,target=/root/.npm<br/>npm install</code>                                                 | <ul style="list-style-type: none"> <li>Mounts a <b>persistent build-time cache</b> at the target path for one <code>RUN</code> only — contents survive between builds but are <b>never stored in image layers</b></li> <li>target the package manager's download dir ( <code>/root/.npm</code> , <code>/var/cache/apt</code> , <code>/root/.cache/pip</code> ), not the install dir.</li> </ul> |
| <b>Build secrets</b>             | <code>RUN --mount=type=secret,id=token<br/>curl -H "Auth: \$(cat /run/secrets/token)"</code>                          | <ul style="list-style-type: none"> <li>Mounts a secret as a tmpfs file (e.g. <code>/run/secrets/token</code> ) for a single <code>RUN</code> ; <b>never written to layers, docker history , or build cache</b></li> <li>replaces the leaky <code>--build-arg / ENV SECRET=...</code> pattern.</li> </ul>                                                                                        |
| <b>Minimal base images</b>       | <code>FROM alpine:3.21<br/>FROM gcr.io/distroless/static<br/>FROM scratch</code>                                      | <ul style="list-style-type: none"> <li>Smaller base = smaller image and smaller attack surface (alpine ~5 MB musl-libc vs ubuntu ~80 MB)</li> <li>distroless drops the shell and package manager; <code>scratch</code> is empty — best for statically-compiled Go/Rust binaries.</li> </ul>                                                                                                     |
| <b>Combining RUN commands</b>    | <code>RUN apt-get update &amp;&amp;<br/>apt-get install -y curl<br/>&amp;&amp; rm -rf<br/>/var/lib/apt/lists/*</code> | <ul style="list-style-type: none"> <li>Chain commands with <code>&amp;&amp;</code> so the install <b>and the cleanup happen in the same layer</b> — files deleted in a later <code>RUN</code> still live in earlier layers' history</li> <li>also prevents the stale- <code>apt-get update</code> caching trap.</li> </ul>                                                                      |
| <b>Buildx for multi-platform</b> | <code>docker buildx build<br/>--platform<br/>linux/amd64,linux/arm64 -t<br/>app .</code>                              | <ul style="list-style-type: none"> <li><code>buildx</code> (the BuildKit-backed CLI) builds for <b>multiple architectures in one invocation</b>, producing a single multi-arch manifest</li> <li>cross-arch builds need either QEMU emulation or native builder nodes per platform.</li> </ul>                                                                                                  |

TABLE 6

## Port Publishing and Networking

Container ports stay isolated from the host until you explicitly publish them. The `-p` family of flags wires a host address and port to a container port; getting the syntax — especially the host-then-container order, the default `0.0.0.0` bind, and the implicit TCP protocol — right is the difference between a reachable service and a silent failure.

| Option                  | Example                                                | Description                                                                                                                                                                                                                                             |
|-------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>-p (publish)</b>     | <code>docker run -p 8080:80<br/>nginx</code>           | <ul style="list-style-type: none"> <li>Maps a <b>host port to a container port</b> using <code>HOST_PORT:CONTAINER_PORT</code></li> <li>with no host IP, binds to <code>0.0.0.0</code> on every interface.</li> </ul>                                   |
| <b>-P (publish-all)</b> | <code>docker run -P nginx</code>                       | <ul style="list-style-type: none"> <li>Publishes every <code>EXPOSE</code> d port to a <b>random ephemeral host port</b></li> <li>query the chosen ports with <code>docker port &lt;container&gt;</code> .</li> </ul>                                   |
| <b>Specific host IP</b> | <code>docker run -p<br/>127.0.0.1:8080:80 nginx</code> | <ul style="list-style-type: none"> <li>Binds the published port to a <b>specific host interface</b></li> <li>use <code>127.0.0.1</code> to restrict access to the host (e.g. behind a reverse proxy).</li> </ul>                                        |
| <b>Port ranges</b>      | <code>docker run -p<br/>8000-8010:8000-8010 app</code> | <ul style="list-style-type: none"> <li>Maps a <b>range of ports</b> one-to-one</li> <li>a range-to-single form like <code>8000-9000:5000</code> picks one free host port from the range.</li> </ul>                                                     |
| <b>UDP ports</b>        | <code>docker run -p 53:53/udp<br/>dns</code>           | <ul style="list-style-type: none"> <li><code>-p</code> publishes <b>TCP by default</b>; append <code>/udp</code> for UDP services like DNS, QUIC, or SIP</li> <li>to publish both, repeat the flag: <code>-p 53:53/tcp -p 53:53/udp</code> .</li> </ul> |

TABLE 7

## Network Modes

Docker networking is driven by a pluggable set of built-in drivers — each one shapes how a container's network namespace is wired up, what it can reach, and how it appears to the rest of the world. Picking the right one is the difference between a quick local test and a setup that survives a multi-host production deployment.

| Mode                    | Example                                            | Description                                                                                                                                                                                                                                                         |
|-------------------------|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>bridge (default)</b> | <code>docker run --network bridge<br/>nginx</code> | <ul style="list-style-type: none"> <li>Containers get private IPs on a virtual L2 bridge with NAT for outbound traffic</li> <li>the <b>default bridge</b> has <b>no name-based DNS</b> between containers — only <b>user-defined</b> bridge networks do.</li> </ul> |

| Mode    | Example                                                                                      | Description                                                                                                                                                                                                                                                       |
|---------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| host    | <code>docker run --network host nginx</code>                                                 | <ul style="list-style-type: none"> <li>Container shares the <b>host's network namespace</b> — no isolation, no NAT, binds host ports directly</li> <li>fastest but largest security blast radius. Linux only (Desktop 4.34+ via VM bridge).</li> </ul>            |
| none    | <code>docker run --network none app</code>                                                   | <ul style="list-style-type: none"> <li>Isolated network namespace with <b>only lo</b> (loopback)</li> <li>no other interfaces, no external connectivity — useful for batch jobs that must not touch the network.</li> </ul>                                       |
| overlay | <code>docker network create -d overlay --attachable my-net</code>                            | <ul style="list-style-type: none"> <li>Multi-host networking over <b>VXLAN</b> for <b>Docker Swarm</b></li> <li>requires <code>docker swarm init</code> even for single-node use; containers on different swarm nodes see each other as if on one LAN.</li> </ul> |
| macvlan | <code>docker network create -d macvlan --subnet=192.168.1.0/24 --o parent=eth0 my-net</code> | <ul style="list-style-type: none"> <li>Each container gets its <b>own MAC</b> and appears as a physical device on the parent NIC's LAN</li> <li>Linux-only; most cloud providers block it (hypervisor MAC-spoof protection).</li> </ul>                           |
| ipvlan  | <code>docker network create -d ipvlan --subnet=192.168.1.0/24 --o parent=eth0 my-net</code>  | <ul style="list-style-type: none"> <li>Like macvlan but containers <b>share the parent's MAC</b> (L2 mode)</li> <li>works where macvlan is blocked, and avoids upstream-switch MAC-table exhaustion at scale.</li> </ul>                                          |

TABLE 8

## Network Management

Docker networking glues containers to each other and to the outside world. The commands here create and manage **user-defined networks** (almost always what you want — they enable automatic container-name DNS, unlike the default `bridge` ), connect and disconnect running containers, and reach back to the host machine from inside a container.

| Command                          | Example                                          | Description                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker network create</b>     | <code>docker network create mynet</code>         | <ul style="list-style-type: none"> <li>Creates a <b>user-defined network</b> — default driver is <code>bridge</code> ; use <code>-d overlay</code> for Swarm or <code>-d macvlan</code> for L2</li> <li>user-defined bridges enable <b>automatic DNS resolution</b> between containers by name (the default <code>bridge</code> network does not).</li> </ul>   |
| <b>docker network ls</b>         | <code>docker network ls</code>                   | <ul style="list-style-type: none"> <li>Lists <b>all networks</b> with <code>NETWORK ID</code> , <code>NAME</code> , <code>DRIVER</code> , and <code>SCOPE</code></li> <li>always includes the three predefined networks <code>bridge</code> , <code>host</code> , and <code>none</code> that ship with the daemon.</li> </ul>                                   |
| <b>docker network connect</b>    | <code>docker network connect mynet web</code>    | <ul style="list-style-type: none"> <li>Attaches a <b>running</b> container to an additional network — containers can be on multiple networks at once</li> <li>on the primary network they resolve each other by name; on secondary networks they must use IP.</li> </ul>                                                                                        |
| <b>docker network disconnect</b> | <code>docker network disconnect mynet web</code> | <ul style="list-style-type: none"> <li>Removes a container from a network <b>while it is still running</b> — no need to stop it first</li> <li>commonly used before <code>docker network rm</code> to free a network that's blocking removal.</li> </ul>                                                                                                        |
| <b>docker network inspect</b>    | <code>docker network inspect mynet</code>        | <ul style="list-style-type: none"> <li>Returns the full JSON config for a network: driver, IPAM <code>Subnet</code> / <code>Gateway</code> , every <b>connected container</b>, IP and MAC addresses, driver options</li> <li>the first stop when containers can see each other by IP but not by name.</li> </ul>                                                |
| <b>docker network rm</b>         | <code>docker network rm mynet</code>             | <ul style="list-style-type: none"> <li>Removes a user-defined network — <b>fails with "has active endpoints"</b> if any container is still attached</li> <li>disconnect or stop+remove the container first, then retry. <code>-f</code> only ignores "no such network", it does not force-detach.</li> </ul>                                                    |
| <b>docker network prune</b>      | <code>docker network prune</code>                | <ul style="list-style-type: none"> <li>Removes every <b>unused</b> user-defined network (no container attached) and prompts for confirmation; <code>-f</code> skips the prompt, <code>--filter "until=24h"</code> narrows by age</li> <li>the predefined <code>bridge</code> , <code>host</code> , and <code>none</code> networks are never touched.</li> </ul> |

| Command                     | Example                                            | Description                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>host.docker.internal</b> | <code>curl http://host.docker.internal:3000</code> | <ul style="list-style-type: none"> <li>Special DNS name that resolves to the <b>host machine</b> from inside a container — built in on Docker Desktop (Mac/Windows/WSL)</li> <li>on Linux Docker Engine it does <b>not</b> exist by default; add <code>--add-host=host.docker.internal:host-gateway</code> to <code>docker run</code> (or <code>extra_hosts</code> in Compose).</li> </ul> |

TABLE 9

## Volume Management

Containers are ephemeral, but most real applications need data that outlives any single container. Docker offers three storage primitives — named volumes, bind mounts, and tmpfs — each with very different durability, portability, and host-coupling trade-offs, plus two CLI shapes ( `-v` and `--mount` ) whose subtle differences trip people up.

| Type                     | Example                                                                      | Description                                                                                                                                                                                                                                                                                                                            |
|--------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Named volumes</b>     | <code>docker run -v data:/app/data nginx</code>                              | <ul style="list-style-type: none"> <li>Docker-managed storage under <code>/var/lib/docker/volumes/</code>, lifecycle independent of the container</li> <li><b>preferred</b> for production; easy to back up, share between containers, and reference by name.</li> </ul>                                                               |
| <b>--mount syntax</b>    | <code>docker run --mount type=volume,source=mydata,target=/data nginx</code> | <ul style="list-style-type: none"> <li><b>Explicit, key-value</b> alternative to <code>-v</code> that supports <code>type=volume</code>, <code>type=bind</code>, and <code>type=tmpfs</code></li> <li>recommended for scripting because it <b>errors on a missing bind source</b> instead of silently creating a directory.</li> </ul> |
| <b>Bind mounts</b>       | <code>docker run -v /host/path:/container/path nginx</code>                  | <ul style="list-style-type: none"> <li>Maps an <b>absolute host path</b> straight into the container</li> <li>changes sync in real time, ideal for development; portability and permission ownership are your problem, not Docker's.</li> </ul>                                                                                        |
| <b>tmpfs mounts</b>      | <code>docker run --tmpfs /tmp nginx</code>                                   | <ul style="list-style-type: none"> <li>Stores data <b>only in host RAM</b>, never written to disk; removed when the container stops</li> <li>good for caches and short-lived secrets, not for anything that must survive a restart.</li> </ul>                                                                                         |
| <b>Anonymous volumes</b> | <code>docker run -v /app/data nginx</code>                                   | <ul style="list-style-type: none"> <li>Created automatically with a <b>random hash name</b> when no source is given</li> <li>persist after the container exits unless you used <code>--rm</code>; accumulate quickly and are hard to identify.</li> </ul>                                                                              |
| <b>Read-only volumes</b> | <code>docker run -v data:/app:ro nginx</code>                                | <ul style="list-style-type: none"> <li><code>:ro</code> (or <code>readonly</code> with <code>--mount</code>) makes the mount <b>read-only inside the container</b></li> <li>the host can still modify the underlying files; it is a container-side guard, not a host-side lock.</li> </ul>                                             |
| <b>volumes-from</b>      | <code>docker run --volumes-from web nginx</code>                             | <ul style="list-style-type: none"> <li>Mounts <b>all volumes</b> from another container at their original paths</li> <li>classic backup/restore trick; modern setups usually share a single named volume by name instead.</li> </ul>                                                                                                   |

TABLE 10

## Volume Commands

Volumes are Docker's managed mechanism for persisting data outside a container's writable layer, so data survives container restarts and deletions. The `docker volume` subcommands let you create, inspect, and clean up that storage — and most of the gotchas live in what they *don't* do (auto-show usage, expose host paths on Docker Desktop, or prune named volumes by default).

| Command                     | Example                                  | Description                                                                                                                                                                                                        |
|-----------------------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker volume create</b> | <code>docker volume create mydata</code> | <ul style="list-style-type: none"> <li>Creates a <b>named volume</b> with a chosen driver and options</li> <li>rarely required for plain use — <code>docker run -v mydata:/path</code> auto-creates it.</li> </ul> |
| <b>docker volume ls</b>     | <code>docker volume ls</code>            | <ul style="list-style-type: none"> <li>Lists all volumes with driver and name</li> <li>does <b>not</b> show which container uses each — pair with <code>docker ps --filter volume=&lt;name&gt;</code>.</li> </ul>  |

| Command                      | Example                                   | Description                                                                                                                                                                                                   |
|------------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker volume inspect</b> | <code>docker volume inspect mydata</code> | <ul style="list-style-type: none"> <li>Shows volume metadata including the <b>Mountpoint</b></li> <li>on <b>Docker Desktop</b>, that path lives inside the utility VM, not on the host filesystem.</li> </ul> |
| <b>docker volume rm</b>      | <code>docker volume rm mydata</code>      | <ul style="list-style-type: none"> <li><b>Removes a volume</b></li> <li>fails if any container references it — including stopped ones (check with <code>docker ps -a</code>).</li> </ul>                      |
| <b>docker volume prune</b>   | <code>docker volume prune</code>          | <ul style="list-style-type: none"> <li>By default removes only unused <b>anonymous</b> volumes</li> <li>use <code>-a / --all</code> to also delete unused named volumes (API 1.42+).</li> </ul>               |

TABLE 11

## Resource Constraints

By default a Docker container can use as much memory and CPU as the host kernel will give it, which makes "noisy neighbor" problems and OOM cascades easy to trigger. These flags map directly onto Linux cgroup controllers (memory, CPU CFS, cpuset, pids, blkio) and let you pick between **hard caps**, **soft reservations**, **relative weights**, and **device pinning** — distinctions that decide whether a container crashes with exit 137 or simply slows down.

| Flag                        | Example                                                             | Description                                                                                                                                                                                                           |
|-----------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>--memory (-m)</b>        | <code>docker run -m 512m nginx</code>                               | <ul style="list-style-type: none"> <li><b>Hard</b> memory limit (cgroup)</li> <li>OOM killer sends SIGKILL on breach (exit 137), suffixes: <code>b k m g</code>.</li> </ul>                                           |
| <b>--memory-reservation</b> | <code>docker run --memory-reservation 256m nginx</code>             | <ul style="list-style-type: none"> <li><b>Soft limit</b> only enforced under host memory pressure</li> <li>must be lower than <code>--memory</code> to take effect; container may exceed it.</li> </ul>               |
| <b>--cpus</b>               | <code>docker run --cpus 1.5 nginx</code>                            | <ul style="list-style-type: none"> <li><b>Hard cap</b> on total CPU time via CFS quota/period</li> <li><code>1.5 = --cpu-period=100000 + --cpu-quota=150000</code>; kernel picks which cores.</li> </ul>              |
| <b>--cpu-shares</b>         | <code>docker run --cpu-shares 512 nginx</code>                      | <ul style="list-style-type: none"> <li><b>Relative weight</b> (default 1024) for CFS — not a cap</li> <li>only honored under CPU contention; idle host = no effect.</li> </ul>                                        |
| <b>--cpuset-cpus</b>        | <code>docker run --cpuset-cpus 0,1 nginx</code>                     | <ul style="list-style-type: none"> <li><b>Pins</b> the container to specific physical CPUs (cores 0 and 1)</li> <li>used for NUMA locality and cache-sensitive workloads.</li> </ul>                                  |
| <b>--pids-limit</b>         | <code>docker run --pids-limit 100 nginx</code>                      | <ul style="list-style-type: none"> <li>Caps total <b>PIDs (processes + threads)</b> via the pids cgroup</li> <li>mitigates fork bombs; thread-heavy JVM/Python apps can hit it.</li> </ul>                            |
| <b>--blkio-weight</b>       | <code>docker run --blkio-weight 500 nginx</code>                    | <ul style="list-style-type: none"> <li><b>Relative</b> block-I/O weight (default 500, range 10-1000)</li> <li>effective only on schedulers that honor it (e.g., BFQ/CFQ) and under disk contention.</li> </ul>        |
| <b>--gpus</b>               | <code>docker run --gpus all nvidia/cuda:12.0-base nvidia-smi</code> | <ul style="list-style-type: none"> <li>Exposes host <b>GPUs</b> to the container</li> <li>requires the <b>NVIDIA Container Toolkit</b> on the host; use <code>"device=0,1"</code> to target specific GPUs.</li> </ul> |

TABLE 12

## Restart Policies

Restart policies tell the Docker daemon when to bring a container back up on its own — after a crash, a host reboot, or a daemon restart. Pick the right one and a single-host stack stays available through reboots; pick the wrong one and a "stopped" container quietly comes back during maintenance.

| Policy              | Example                                              | Description                                                                                                                                                          |
|---------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>no (default)</b> | <code>docker run --restart no nginx</code>           | <ul style="list-style-type: none"> <li><b>Never restart</b> container automatically</li> <li>manual <code>docker start</code> still works.</li> </ul>                |
| <b>on-failure</b>   | <code>docker run --restart on-failure:3 nginx</code> | <ul style="list-style-type: none"> <li>Restarts only on <b>non-zero exit code</b></li> <li>optional <code>:N</code> cap; not triggered by daemon restart.</li> </ul> |

| Policy                | Example                                                | Description                                                                                                                                                                                  |
|-----------------------|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>always</b>         | <code>docker run --restart always nginx</code>         | <ul style="list-style-type: none"> <li>• <b>Always restarts</b>, including after daemon restart</li> <li>• brings back even manually-stopped containers on reboot.</li> </ul>                |
| <b>unless-stopped</b> | <code>docker run --restart unless-stopped nginx</code> | <ul style="list-style-type: none"> <li>• Like <code>always</code> but <b>respects manual stops</b> across daemon restarts</li> <li>• preferred for single-host production stacks.</li> </ul> |

TABLE 13

## Health Checks

Docker can probe a running container with a periodic command and surface a `healthy` / `unhealthy` / `starting` state on top of the normal `running` state. Set the probe in the image via the `HEALTHCHECK` Dockerfile instruction, or attach/override it at runtime with `docker run --health-*` flags. The health state shows up in `docker ps` and `docker inspect`, but **Docker itself does not restart unhealthy containers** — that's the orchestrator's (Swarm/Compose with `condition: service_healthy` / Kubernetes) job.

| Option                         | Example                                                                                     | Description                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>HEALTHCHECK instruction</b> | <code>HEALTHCHECK --interval=30s<br/>CMD curl -f<br/>http://localhost/<br/>   exit 1</code> | <ul style="list-style-type: none"> <li>• Dockerfile instruction defining the probe baked into the image</li> <li>• exit <code>0</code> = healthy, <code>1</code> = unhealthy, <code>2</code> is reserved</li> <li>• only the last <code>HEALTHCHECK</code> in a Dockerfile takes effect; <code>HEALTHCHECK NONE</code> disables one inherited from the base image.</li> </ul> |
| <b>--health-cmd</b>            | <code>docker run --health-cmd<br/>"curl -f<br/>http://localhost/" nginx</code>              | <ul style="list-style-type: none"> <li>• Runtime override that attaches (or replaces) the probe command without rebuilding the image</li> <li>• useful for ad-hoc testing or images shipped without a <code>HEALTHCHECK</code>.</li> </ul>                                                                                                                                    |
| <b>--health-interval</b>       | <code>docker run<br/>--health-interval=10s<br/>nginx</code>                                 | <ul style="list-style-type: none"> <li>• Wait time <b>between</b> checks (after the previous one completes), default <code>30s</code></li> <li>• the first check also runs one interval after container start.</li> </ul>                                                                                                                                                     |
| <b>--health-timeout</b>        | <code>docker run<br/>--health-timeout=5s nginx</code>                                       | <ul style="list-style-type: none"> <li>• Max time for a single check to complete, default <code>30s</code></li> <li>• if exceeded, the probe is <code>SIGKILL</code>-ed and that run counts as a failure.</li> </ul>                                                                                                                                                          |
| <b>--health-retries</b>        | <code>docker run<br/>--health-retries=3 nginx</code>                                        | <ul style="list-style-type: none"> <li>• Number of <b>consecutive</b> failures needed to flip the state to <code>unhealthy</code>, default <code>3</code></li> <li>• one successful check resets the failing streak to zero.</li> </ul>                                                                                                                                       |
| <b>--health-start-period</b>   | <code>docker run<br/>--health-start-period=60s<br/>nginx</code>                             | <ul style="list-style-type: none"> <li>• Initialization grace window, default <code>0s</code> — checks still run during it, but failures do <b>not</b> count toward <code>--health-retries</code></li> <li>• any single success ends the start period early.</li> </ul>                                                                                                       |

TABLE 14

## Docker Compose Services

How each `services:` entry is configured — the attributes that decide what runs, how it reaches the network, what it depends on, and how the platform treats it. Most startup-order, port, and resource bugs in Compose stacks trace back to misreading one of these keys.

| Attribute      | Example                                                            | Description                                                                                                                                                                                                     |
|----------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>image</b>   | <code>image: nginx:alpine</code>                                   | <ul style="list-style-type: none"> <li>• Names the <b>image to run</b></li> <li>• pulled from the registry if not present locally.</li> </ul>                                                                   |
| <b>build</b>   | <code>build:<br/>context: .<br/>dockerfile: Dockerfile.prod</code> | <ul style="list-style-type: none"> <li>• Builds an image from a <b>Dockerfile</b></li> <li>• <code>context</code> is resolved <b>relative to the Compose file's directory</b>, not the shell's cwd.</li> </ul>  |
| <b>ports</b>   | <code>ports:<br/>- "8080:80"</code>                                | <ul style="list-style-type: none"> <li>• Publishes ports as <code>"HOST:CONTAINER"</code></li> <li>• <b>always quote</b> short syntax — YAML parses unquoted <code>xx:yy</code> as a base-60 number.</li> </ul> |
| <b>volumes</b> | <code>volumes:<br/>- ./data:/data:ro</code>                        | <ul style="list-style-type: none"> <li>• Mounts volumes or bind mounts</li> <li>• trailing <code>:ro</code> mounts the path <b>read-only inside the container</b>.</li> </ul>                                   |

| Attribute          | Example                                                                                                                     | Description                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>environment</b> | <code>environment:<br/>NODE_ENV: production</code>                                                                          | <ul style="list-style-type: none"> <li>Sets <b>runtime environment variables</b></li> <li>only Compose's <code>\${VAR}</code> interpolation expands — a literal <code>\$BAR</code> is passed through, not shell-expanded.</li> </ul>                                                                                                                                                                                       |
| <b>depends_on</b>  | <code>depends_on:<br/>db:<br/>condition: service_healthy<br/>restart: true<br/>required: false</code>                       | <ul style="list-style-type: none"> <li>Controls <b>startup order</b>; default <code>service_started</code> only waits for the container to start</li> <li><code>service_healthy</code> waits for the dependency's healthcheck to pass</li> <li><code>restart: true</code> restarts the dependent if the dependency restarts; <code>required: false</code> allows startup even if the dependency is unavailable.</li> </ul> |
| <b>networks</b>    | <code>networks:<br/>- frontend</code>                                                                                       | <ul style="list-style-type: none"> <li>Attaches the service to <b>named networks</b></li> <li>inside a shared network, services reach each other by service name on the container port.</li> </ul>                                                                                                                                                                                                                         |
| <b>restart</b>     | <code>restart: unless-stopped</code>                                                                                        | <ul style="list-style-type: none"> <li>Sets the container <b>restart policy</b></li> <li>options: <code>no</code> (default), <code>always</code>, <code>on-failure[:N]</code>, <code>unless-stopped</code>.</li> </ul>                                                                                                                                                                                                     |
| <b>command</b>     | <code>command: npm run dev</code>                                                                                           | <ul style="list-style-type: none"> <li><b>Overrides the image's CMD</b> only</li> <li>the image's <code>ENTRYPOINT</code> still runs and receives this as its arguments.</li> </ul>                                                                                                                                                                                                                                        |
| <b>entrypoint</b>  | <code>entrypoint:<br/>/app/entrypoint.sh</code>                                                                             | <ul style="list-style-type: none"> <li><b>Replaces the image's ENTRYPOINT</b> entirely</li> <li>setting it does not clear the image's <code>CMD</code>, which is still passed as arguments.</li> </ul>                                                                                                                                                                                                                     |
| <b>healthcheck</b> | <code>healthcheck:<br/>test: ["CMD", "curl", "-f",<br/>"http://localhost/"]<br/>interval: 30s<br/>start_interval: 2s</code> | <ul style="list-style-type: none"> <li>Defines a container <b>health probe</b></li> <li><code>CMD</code> execs the args directly; <code>CMD-SHELL</code> wraps in <code>/bin/sh -c</code> so operators like <code>  </code> work</li> <li><code>start_interval</code> sets a faster check rate during <code>start_period</code> so dependent services start sooner.</li> </ul>                                             |
| <b>deploy</b>      | <code>deploy:<br/>resources:<br/>limits:<br/>memory: 512M</code>                                                            | <ul style="list-style-type: none"> <li>Most keys ( <code>replicas</code>, <code>placement</code>, <code>update_config</code> ) are <b>Swarm-only</b> and ignored by <code>docker compose up</code></li> <li><code>deploy.resources</code> for limits and GPU reservations does apply to plain Compose.</li> </ul>                                                                                                          |

TABLE 15

## Docker Compose Commands

The Compose CLI manages multi-container apps defined in a `compose.yaml` file as a single **Compose project** — a scope Compose tracks by project name. Note: the legacy `docker-compose` (v1, hyphenated) was permanently removed in April 2025; only `docker compose` (space, no hyphen) works. Knowing what each command actually mutates (containers, networks, volumes, images) is the difference between a clean restart and accidentally deleting your database.

| Command                     | Example                                      | Description                                                                                                                                                                                                                                         |
|-----------------------------|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker compose up</b>    | <code>docker compose up -d</code>            | <ul style="list-style-type: none"> <li><b>Builds, (re)creates, starts</b> all services and their networks</li> <li><code>-d</code> detaches; recreates containers whose config changed since last <code>up</code>.</li> </ul>                       |
| <b>docker compose down</b>  | <code>docker compose down -v</code>          | <ul style="list-style-type: none"> <li>Stops and removes containers and the project's networks</li> <li>Keeps named volumes by default; <code>-v</code> <b>also removes named volumes</b> (data loss).</li> </ul>                                   |
| <b>docker compose ps</b>    | <code>docker compose ps</code>               | <ul style="list-style-type: none"> <li>Lists <b>running</b> containers for the current Compose project</li> <li>add <code>-a</code> for stopped ones.</li> </ul>                                                                                    |
| <b>docker compose logs</b>  | <code>docker compose logs -f web</code>      | <ul style="list-style-type: none"> <li>Shows aggregated <b>logs from services</b></li> <li><code>-f</code> follows; pass a service name to filter to one.</li> </ul>                                                                                |
| <b>docker compose exec</b>  | <code>docker compose exec web sh</code>      | <ul style="list-style-type: none"> <li>Runs a command inside a <b>running service container</b></li> <li>allocates a TTY by default; <code>--index</code> picks a replica.</li> </ul>                                                               |
| <b>docker compose build</b> | <code>docker compose build --no-cache</code> | <ul style="list-style-type: none"> <li><b>Builds or rebuilds service images</b> from each service's <code>build</code>:</li> <li><code>--no-cache</code> discards the build cache; pair with <code>--pull</code> to refresh base images.</li> </ul> |
| <b>docker compose pull</b>  | <code>docker compose pull</code>             | <ul style="list-style-type: none"> <li>Pulls images for services from their registries</li> <li><b>does not start containers.</b></li> </ul>                                                                                                        |

| Command                       | Example                                           | Description                                                                                                                                                                                                                                                                    |
|-------------------------------|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker compose restart</b> | <code>docker compose restart web</code>           | Restarts the existing containers <b>without re-reading</b> the Compose file — config edits do not apply (use <code>up -d</code> instead).                                                                                                                                      |
| <b>docker compose stop</b>    | <code>docker compose stop</code>                  | <ul style="list-style-type: none"> <li>Stops services <b>without removing containers, networks, or volumes</b></li> <li>faster than <code>down</code> ; resume with <code>start</code> .</li> </ul>                                                                            |
| <b>docker compose start</b>   | <code>docker compose start</code>                 | <ul style="list-style-type: none"> <li><b>Starts previously stopped containers</b> for the project</li> <li>never creates new containers — use <code>up</code> for that.</li> </ul>                                                                                            |
| <b>docker compose run</b>     | <code>docker compose run --rm web npm test</code> | <ul style="list-style-type: none"> <li>Runs a <b>one-off command</b> in a new container based on the service</li> <li><code>--rm</code> removes it after exit; starts linked services unless <code>--no-deps</code> .</li> </ul>                                               |
| <b>docker compose watch</b>   | <code>docker compose watch</code>                 | <ul style="list-style-type: none"> <li>Reacts to <b>file changes</b> declared under <code>develop.watch</code></li> <li>actions: <code>sync</code> , <code>rebuild</code> , <code>sync+restart</code> (modern replacement for <code>bind-mount-and-restart</code>).</li> </ul> |

TABLE 16

## Docker Compose Advanced Features

Compose's advanced primitives go beyond a basic `compose.yaml` : profiles toggle optional services per environment, `extends` / `include` / `override` files compose larger stacks out of smaller pieces, `secrets` and `configs` mount files instead of leaking env vars, `develop.watch` keeps containers in sync with source changes, and the `models` top-level element declares AI/ML model dependencies alongside services.

| Feature                                                | Example                                                                                                                               | Description                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>profiles</b>                                        | <code>profiles: [debug]</code>                                                                                                        | <ul style="list-style-type: none"> <li>Groups services for <b>selective activation</b></li> <li><code>docker compose --profile debug up</code> only starts debug services.</li> </ul>                                                                                                                                                                                                                                       |
| <b>extends</b>                                         | <code>extends:<br/>file: common.yml<br/>service: base</code>                                                                          | <ul style="list-style-type: none"> <li><b>Inherits configuration</b> from another service</li> <li>enables reusable base definitions.</li> </ul>                                                                                                                                                                                                                                                                            |
| <b>override files</b>                                  | <code>docker-compose.override.yml</code>                                                                                              | <ul style="list-style-type: none"> <li>Automatically merged with <code>docker-compose.yml</code></li> <li><b>environment-specific overrides</b> without modifying base file.</li> </ul>                                                                                                                                                                                                                                     |
| <b>include</b>                                         | <code>include:<br/>- monitoring.yml<br/>- oci://docker.io/org/stack:<br/>latest</code>                                                | <ul style="list-style-type: none"> <li>Includes <b>separate Compose files or OCI artifacts</b> to combine projects</li> <li>⚠ update to v2.40.2+ if using OCI includes — CVE-2025-62725 path-traversal fix.</li> </ul>                                                                                                                                                                                                      |
| <b>env_file</b>                                        | <code>env_file:<br/>- .env.prod</code>                                                                                                | <ul style="list-style-type: none"> <li>Loads environment variables from <b>files</b></li> <li>keeps secrets out of Compose file.</li> </ul>                                                                                                                                                                                                                                                                                 |
| <b>secrets</b>                                         | <code>secrets:<br/>- db_password</code>                                                                                               | <ul style="list-style-type: none"> <li>Injects secrets as <b>files</b> in container</li> <li>mounted at <code>/run/secrets/</code> .</li> </ul>                                                                                                                                                                                                                                                                             |
| <b>configs</b>                                         | <code>configs:<br/>- nginx.conf</code>                                                                                                | <ul style="list-style-type: none"> <li>Like secrets but for <b>non-sensitive config files</b></li> <li>also mounted as files.</li> </ul>                                                                                                                                                                                                                                                                                    |
| <b>develop (watch)</b>                                 | <code>develop:<br/>watch:<br/>- action: sync<br/>path: ./src<br/>target: /app/src<br/>- action: rebuild<br/>path: package.json</code> | <ul style="list-style-type: none"> <li>Configures <b>hot-reload</b> for development (formerly <code>x-develop</code> — drop the <code>x-</code> prefix)</li> <li>actions: <code>sync</code> (copy files), <code>rebuild</code> (rebuild image), <code>sync+restart</code> (copy and restart)</li> <li><code>initial_sync</code> syncs all matching files immediately on <code>docker compose watch</code> start.</li> </ul> |
| <b>Lifecycle hooks (pre_start/post_start/pre_stop)</b> | <code>services:<br/>app:<br/>pre_start:<br/>- command: ./migrate.sh</code>                                                            | <ul style="list-style-type: none"> <li><code>pre_start</code> (Compose v5.3+) runs one or more <b>ephemeral init containers</b> to completion, in order, before the service container starts, and fails the bring-up if any step exits non-zero</li> <li><code>post_start</code> / <code>pre_stop</code> instead run a command inside the already-running service container at those points.</li> </ul>                     |

| Feature                    | Example                                                                                                                                    | Description                                                                                                                                                                                                                                                                                                                                  |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>models (Compose AI)</b> | <pre>models:   llm:     model: ai/smollm2     context_size: 4096</pre>                                                                     | <ul style="list-style-type: none"><li>• Top-level element declaring <b>AI/ML model dependencies</b> as OCI artifacts alongside services</li><li>• platform (e.g. Docker Model Runner) injects <code>&lt;NAME&gt;_URL</code> and <code>&lt;NAME&gt;_MODEL</code> env vars into consuming services; portable across cloud platforms.</li></ul> |
| <b>YAML anchors</b>        | <pre>x-common: &amp;common restart: always services:   web:     &lt;&lt;: *common</pre>                                                    | <ul style="list-style-type: none"><li>• Reuses config blocks with <b>anchors and aliases</b></li><li>• <code>x-</code> prefix defines extension fields that Compose ignores.</li></ul>                                                                                                                                                       |
| <b>GPU support</b>         | <pre>deploy:   resources:     reservations:       devices:         - driver: nvidia           count: 1           capabilities: [gpu]</pre> | <ul style="list-style-type: none"><li>• Requests <b>GPU access</b> for a service (NVIDIA and AMD supported)</li><li>• requires NVIDIA Container Toolkit or AMD ROCm driver on host.</li></ul>                                                                                                                                                |

TABLE 17

## Registry and Authentication

Authenticating to a Docker registry sets up the credentials that later `push` and `pull` commands silently reuse, so the way you log in directly shapes how exposed those secrets are. This table covers the day-to-day commands and the two safer non-interactive patterns to prefer over `-p`.

| Command                        | Example                                                         | Description                                                                                                                                                                                                                                                                                    |
|--------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker login</b>            | <pre>docker login registry.example.com</pre>                    | <ul style="list-style-type: none"><li>• <b>Authenticates to a registry</b> and caches credentials for later <code>push</code> / <code>pull</code></li><li>• stores them via the configured credential helper, or base64-encoded in <code>~/.docker/config.json</code> as a fallback.</li></ul> |
| <b>docker logout</b>           | <pre>docker logout registry.example.com</pre>                   | <ul style="list-style-type: none"><li>• Removes the <b>locally stored credential</b> for the given server (or the default registry)</li><li>• does <b>not</b> revoke the token on the registry — rotate it there if it may have leaked.</li></ul>                                              |
| <b>Username/password login</b> | <pre>docker login -u user -p pass</pre>                         | <ul style="list-style-type: none"><li>• Non-interactive login with the password as an argument</li><li>• <b>avoid</b> — leaks into shell history and <code>ps</code>; Docker prints <b>WARNING! Using --password via the CLI is insecure</b>.</li></ul>                                        |
| <b>Token authentication</b>    | <pre>echo \$TOKEN   docker login -u user --password-stdin</pre> | <ul style="list-style-type: none"><li>• Reads the password or PAT from <b>stdin</b> so it never appears in history or process listings</li><li>• the recommended pattern for CI and short-lived cloud tokens (ECR, GCR, ACR).</li></ul>                                                        |

TABLE 18

## Image Tagging Strategies

A Docker tag is just a mutable, human-readable pointer — `docker tag a b` creates an alias to the same image ID, and any tag (including `latest`) can be reassigned later. Choosing a tagging convention is really choosing how much version, source, and environment information you bake into that pointer so future-you can answer "what is actually running?" without guessing.

| Strategy                   | Example                     | Description                                                                                                                                        |
|----------------------------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Semantic versioning</b> | <pre>myapp:1.2.3</pre>      | <ul style="list-style-type: none"><li>• Uses <b>major.minor.patch</b> format</li><li>• best practice for release tracking and rollbacks.</li></ul> |
| <b>Git commit SHA</b>      | <pre>myapp:a3f2c1b</pre>    | <ul style="list-style-type: none"><li>• Tags with <b>Git commit hash</b></li><li>• enables exact source code traceability.</li></ul>               |
| <b>Environment tags</b>    | <pre>myapp:prod-1.2.3</pre> | <ul style="list-style-type: none"><li>• Includes <b>environment prefix</b></li><li>• clearly indicates deployment target.</li></ul>                |

| Strategy               | Example                         | Description                                                                                                                                    |
|------------------------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Date-based tags</b> | <code>myapp:2026-03-01</code>   | <ul style="list-style-type: none"> <li>Timestamps with <b>YYYY-MM-DD</b></li> <li>useful for nightly builds or time-based releases.</li> </ul> |
| <b>Branch tags</b>     | <code>myapp:feature-auth</code> | <ul style="list-style-type: none"> <li>Tags with <b>branch name</b></li> <li>tracks feature development images.</li> </ul>                     |

TABLE 19

## Cleanup and Maintenance

Cleanup commands reclaim disk space, but each one targets a different scope and several carry real data-loss risk if used carelessly. Knowing what each `prune` does and does not touch — especially `--volumes`, `-a`, and the `until=` filter — is the difference between a tidy host and an outage.

| Command                       | Example                                                  | Description                                                                                                                                                                                                                                                                                     |
|-------------------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker system prune</b>    | <code>docker system prune -a</code>                      | <ul style="list-style-type: none"> <li>Removes stopped containers, unused networks, <b>dangling images</b>, and unused build cache</li> <li><code>-a</code> also removes any image not used by a container; volumes are <b>NOT</b> included unless <code>--volumes</code> is passed.</li> </ul> |
| <b>docker image prune</b>     | <code>docker image prune -a --filter "until=168h"</code> | <ul style="list-style-type: none"> <li>Without <code>-a</code> removes only <b>dangling images</b> (untagged, <code>&lt;none&gt;</code>)</li> <li><code>-a</code> removes all unused images; <code>--filter "until=168h"</code> keeps images created in the <b>last 7 days</b>.</li> </ul>      |
| <b>docker container prune</b> | <code>docker container prune</code>                      | <ul style="list-style-type: none"> <li>Removes all <b>stopped containers</b> and their writable layers</li> <li>running containers are untouched; prompts before deletion unless <code>-f</code>.</li> </ul>                                                                                    |
| <b>docker volume prune</b>    | <code>docker volume prune</code>                         | <ul style="list-style-type: none"> <li>Deletes <b>unused volumes</b> (any volume not attached to a container)</li> <li><b>Data-loss risk</b> — back up first, and remember <code>docker system prune</code> does NOT touch volumes by default.</li> </ul>                                       |
| <b>docker network prune</b>   | <code>docker network prune</code>                        | <ul style="list-style-type: none"> <li>Removes <b>unused custom networks</b></li> <li>built-in networks ( <code>bridge</code>, <code>host</code>, <code>none</code> ) are never pruned.</li> </ul>                                                                                              |
| <b>docker builder prune</b>   | <code>docker builder prune</code>                        | <ul style="list-style-type: none"> <li>Clears the <b>BuildKit build cache</b> for the active builder</li> <li>does not touch images or containers; next build rebuilds layers from scratch.</li> </ul>                                                                                          |

TABLE 20

## Logging Drivers

Docker streams every container's stdout/stderr through a logging driver, and the choice between the default `json-file` driver and alternatives like `local`, `journald`, or remote forwarders shapes disk usage, queryability, and how logs reach a central pipeline.

| Driver                     | Example                                                                  | Description                                                                                                                                                                                                                                                                                 |
|----------------------------|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>json-file (default)</b> | <code>--log-driver json-file</code>                                      | <ul style="list-style-type: none"> <li>Stores logs as <b>JSON</b> on disk</li> <li>queryable with <code>docker logs</code>, <b>no rotation by default</b> — set <code>max-size</code> / <code>max-file</code> to avoid filling <code>/var/lib/docker</code>.</li> </ul>                     |
| <b>local</b>               | <code>--log-driver local</code>                                          | <ul style="list-style-type: none"> <li>Optimized <b>local file</b> driver with automatic log rotation and compression by default</li> <li>Docker's recommended local driver; now supports <code>label</code>, <code>env</code>, and <code>tag</code> log options (Engine 29.5+).</li> </ul> |
| <b>syslog</b>              | <code>--log-driver syslog --log-opt syslog-address=udp://host:514</code> | <ul style="list-style-type: none"> <li>Forwards logs to a <b>syslog daemon</b> over UDP/TCP/TLS</li> <li>integrates with centralized logging.</li> </ul>                                                                                                                                    |
| <b>journald</b>            | <code>--log-driver journald</code>                                       | <ul style="list-style-type: none"> <li>Sends logs to the <b>systemd journal</b> (Linux only)</li> <li>query with <code>journalctl CONTAINER_NAME=foo</code>.</li> </ul>                                                                                                                     |
| <b>fluentd</b>             | <code>--log-driver fluentd --log-opt fluentd-address=host:24224</code>   | <ul style="list-style-type: none"> <li>Forwards to a <b>Fluentd</b> collector</li> <li>popular for Kubernetes and centralized logging.</li> </ul>                                                                                                                                           |

| Driver         | Example                                                                                            | Description                                                                                                                                                                                                |
|----------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>awslogs</b> | <code>--log-driver awslogs --log-opt awslogs-group=myapp</code>                                    | <ul style="list-style-type: none"> <li>Sends logs to <b>AWS CloudWatch Logs</b></li> <li>credentials come from the daemon/host (EC2 instance profile or ECS task role), not container env vars.</li> </ul> |
| <b>gcplogs</b> | <code>--log-driver gcplogs</code>                                                                  | <ul style="list-style-type: none"> <li>Forwards to <b>Google Cloud Logging</b></li> <li>auto-detects project, zone, and instance from the GCE metadata server.</li> </ul>                                  |
| <b>splunk</b>  | <code>--log-driver splunk --log-opt splunk-token=XXX --log-opt splunk-url=https://host:8088</code> | <ul style="list-style-type: none"> <li>Sends logs to the <b>Splunk HTTP Event Collector</b></li> <li>enterprise logging solution.</li> </ul>                                                               |

TABLE 21

## Security Practices

Container security is layered defense: drop privileges, shrink the attack surface, and constrain what the kernel will let the container do. These practices stack — running as non-root, dropping capabilities, applying a seccomp profile, and starting from a minimal base image each close a different escape path, and together they're what stands between a single application bug and host compromise.

| Practice                            | Example                                                                 | Description                                                                                                                                                                                                                                                               |
|-------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Run as non-root user</b>         | <code>USER node</code>                                                  | <ul style="list-style-type: none"> <li>Creates and switches to <b>non-root user</b></li> <li>prevents privilege escalation attacks.</li> </ul>                                                                                                                            |
| <b>Read-only root filesystem</b>    | <code>docker run --read-only nginx</code>                               | <ul style="list-style-type: none"> <li>Makes container filesystem <b>immutable</b></li> <li>prevents tampering; use tmpfs for writable dirs.</li> </ul>                                                                                                                   |
| <b>Drop capabilities</b>            | <code>docker run --cap-drop ALL --cap-add NET_BIND_SERVICE nginx</code> | <ul style="list-style-type: none"> <li>Removes <b>Linux capabilities</b></li> <li>principle of least privilege — drop ALL then add only what's needed.</li> </ul>                                                                                                         |
| <b>Use secrets</b>                  | <code>RUN --mount=type=secret,id=token</code>                           | <ul style="list-style-type: none"> <li>Never hardcode secrets in images</li> <li>use <b>BuildKit secrets</b> or runtime secrets.</li> </ul>                                                                                                                               |
| <b>Scan images</b>                  | <code>docker scout cves nginx:latest</code>                             | <ul style="list-style-type: none"> <li>Scans for <b>vulnerabilities</b></li> <li>integrates with Docker Scout, Trivy, Snyk.</li> </ul>                                                                                                                                    |
| <b>Minimal base images</b>          | <code>FROM gcr.io/distroless/static</code>                              | <ul style="list-style-type: none"> <li>Use <b>distroless</b> or alpine</li> <li>fewer packages = smaller attack surface.</li> </ul>                                                                                                                                       |
| <b>No privileged mode</b>           | Avoid <code>--privileged</code>                                         | <ul style="list-style-type: none"> <li><b>Never use</b> in production</li> <li>gives full host access, defeats containerization.</li> </ul>                                                                                                                               |
| <b>Security profiles</b>            | <code>--security-opt seccomp=profile.json</code>                        | <ul style="list-style-type: none"> <li>Apply <b>seccomp/AppArmor/SELinux</b> profiles</li> <li>restricts syscalls and actions.</li> </ul>                                                                                                                                 |
| <b>--no-new-privileges</b>          | <code>docker run --security-opt no-new-privileges nginx</code>          | <ul style="list-style-type: none"> <li>Prevents processes from <b>gaining additional privileges</b> via setuid/setgid</li> <li>critical for defense in depth.</li> </ul>                                                                                                  |
| <b>Rootless mode</b>                | <code>dockerd-rootless-setup tool .sh install</code>                    | <ul style="list-style-type: none"> <li>Runs Docker daemon <b>without root privileges</b></li> <li>mitigates potential vulnerabilities in daemon and container runtime; Engine 29.5+ uses <code>gvisor-tap-vsock</code> as the default rootless network driver.</li> </ul> |
| <b>Docker Hardened Images (DHI)</b> | <code>FROM dhi.io/python:3.12</code>                                    | <ul style="list-style-type: none"> <li>Docker's <b>distroless-based, CVE-free</b> base images</li> <li>free since Dec 2025, auto-patched within 24hrs, include SBOM and provenance attestations; manage via <code>docker dhi</code> CLI.</li> </ul>                       |

TABLE 22

## Signal Handling

How Docker delivers Unix signals to your container's PID 1 — and why a misplaced shell wrapper or a too-short grace period quietly turns "graceful shutdown" into data corruption. These rows cover the signals Docker actually sends, who receives them, and the form choices that decide whether your app ever gets a chance to clean up.

| Signal                         | Example                                                 | Description                                                                                                                                                                                                                                                                                          |
|--------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SIGTERM</b>                 | Sent by <code>docker stop</code>                        | <ul style="list-style-type: none"> <li>• <b>Graceful shutdown</b> signal delivered to the container's PID 1</li> <li>• after a grace period (default 10s on Linux, 30s on Windows), Docker follows up with <code>SIGKILL</code>. Tune with <code>-t</code> / <code>--stop-timeout</code>.</li> </ul> |
| <b>SIGKILL</b>                 | Sent after grace period, or by <code>docker kill</code> | <ul style="list-style-type: none"> <li>• <b>Force kill</b> signal — kernel-enforced, <b>cannot be caught, blocked, or ignored</b></li> <li>• immediate termination with no cleanup.</li> </ul>                                                                                                       |
| <b>SIGINT</b>                  | Ctrl+C in attached mode                                 | <ul style="list-style-type: none"> <li>• <b>Interrupt signal</b> sent to PID 1</li> <li>• most apps treat it like <code>SIGTERM</code> and exit, so Ctrl+C while attached typically stops the container.</li> </ul>                                                                                  |
| <b>SIGHUP</b>                  | <code>docker kill -s SIGHUP web</code>                  | <ul style="list-style-type: none"> <li>• Hup signal — historically "controlling terminal closed"</li> <li>• many daemons (nginx, HAProxy) repurpose it to <b>reload configuration</b> without restarting.</li> </ul>                                                                                 |
| <b>Exec form vs shell form</b> | <code>CMD ["app"]</code> vs <code>CMD app</code>        | <ul style="list-style-type: none"> <li>• <b>Exec form</b> (JSON array): the app IS PID 1 and receives signals directly</li> <li>• shell form wraps in <code>/bin/sh -c</code>, sh becomes PID 1 and does <b>not</b> forward signals — the app never sees <code>SIGTERM</code>.</li> </ul>            |

TABLE 23

## Context and Remote Docker

A Docker context bundles the endpoint URL, TLS material, and name of one daemon so a single `docker` CLI can manage many daemons — local Docker Desktop, an SSH-reachable production server, an ECS endpoint — and switch between them. SSH-based contexts are the canonical secure pattern for remote control because they reuse your SSH keys and avoid exposing the Docker daemon socket on a public TCP port.

| Command                      | Example                                                                 | Description                                                                                                                                                                                                                                                                |
|------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker context create</b> | <code>docker context create prod --docker "host=ssh://user@host"</code> | <ul style="list-style-type: none"> <li>• Registers a <b>named connection</b> to a Docker daemon (local, <code>tcp://</code>, or <code>ssh://</code>)</li> <li>• metadata is stored under <code>~/.docker/contexts/</code>.</li> </ul>                                      |
| <b>docker context use</b>    | <code>docker context use prod</code>                                    | <ul style="list-style-type: none"> <li>• Sets the <b>active context</b> in <code>~/.docker/config.json</code></li> <li>• persists across shells until changed or overridden by <code>DOCKER_HOST</code> / <code>DOCKER_CONTEXT</code> / <code>--context</code>.</li> </ul> |
| <b>docker context ls</b>     | <code>docker context ls</code>                                          | <ul style="list-style-type: none"> <li>• Lists every context with its <code>DOCKER_ENDPOINT</code> URL</li> <li>• the <b>active one is marked with *</b>.</li> </ul>                                                                                                       |
| <b>docker context rm</b>     | <code>docker context rm prod</code>                                     | <ul style="list-style-type: none"> <li>• Removes a context</li> <li>• <b>fails on the active context</b> unless you pass <code>-f</code> / <code>--force</code> or <code>docker context use default</code> first.</li> </ul>                                               |
| <b>DOCKER_HOST variable</b>  | <code>export DOCKER_HOST=ssh://user@host</code>                         | <ul style="list-style-type: none"> <li>• Per-shell daemon-socket override</li> <li>• <b>beats the active context</b> but is itself overridden by <code>DOCKER_CONTEXT</code> and <code>--host</code> / <code>--context</code>.</li> </ul>                                  |

TABLE 24

## Docker Swarm Basics

Swarm mode is the orchestrator built directly into the Docker Engine — no separate install needed. One `docker swarm init` turns the current engine into a single-node cluster (a manager), and additional engines join as workers or managers via a join token. From there you create **services** (replicated containers spread across the cluster) and **stacks** (a whole Compose file deployed as a unit), and the managers schedule tasks, run rolling updates, and keep the desired replica count alive on the routing mesh.

| Command                      | Example                                                          | Description                                                                                                                                                                                                                                                    |
|------------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker swarm init</b>     | <code>docker swarm init</code>                                   | <ul style="list-style-type: none"> <li>Turns the current Docker Engine into a single-node <b>Swarm cluster</b></li> <li>the current node becomes the leader <b>manager</b>.</li> </ul>                                                                         |
| <b>docker swarm join</b>     | <code>docker swarm join --token TOKEN host:2377</code>           | <ul style="list-style-type: none"> <li>Joins a node to an existing Swarm</li> <li>the token used (worker or manager) determines the role of the joining node.</li> </ul>                                                                                       |
| <b>docker service create</b> | <code>docker service create --replicas 3 --name web nginx</code> | <ul style="list-style-type: none"> <li>Creates a new Swarm <b>service</b> distributed across the cluster</li> <li><code>--replicas N</code> declares the desired task count the schedulers keep alive.</li> </ul>                                              |
| <b>docker service scale</b>  | <code>docker service scale web=5</code>                          | <ul style="list-style-type: none"> <li>Scales an <b>already-running</b> replicated service to a new replica count</li> <li>cannot be used on <code>global</code> -mode services.</li> </ul>                                                                    |
| <b>docker service update</b> | <code>docker service update --image nginx:1.21 web</code>        | <ul style="list-style-type: none"> <li>Applies a <b>rolling update</b> — by default updates 1 task at a time, controlled by <code>--update-parallelism</code> and <code>--update-delay</code></li> <li>supports <code>--rollback</code> on failure.</li> </ul> |
| <b>docker node ls</b>        | <code>docker node ls</code>                                      | <ul style="list-style-type: none"> <li>Lists every node the Swarm manager knows about, with <b>status, availability, and manager status</b></li> <li>only runs on a manager node.</li> </ul>                                                                   |
| <b>docker stack deploy</b>   | <code>docker stack deploy -c compose.yml myapp</code>            | <ul style="list-style-type: none"> <li>Deploys a Compose file (v3.0+) as a Swarm <b>stack</b> — creating its services, networks, and volumes</li> <li><code>build:</code> is ignored — images must already exist in a registry.</li> </ul>                     |

TABLE 25

## Environment Variables

Docker lets you inject configuration into containers through several distinct channels — runtime CLI flags, files, Dockerfile baked-in values, build-time-only arguments, and Compose declarations. Picking the right one matters: some persist into the image (visible in `docker history`), some live only during the build, and none of them are a safe place for secrets.

| Method                     | Example                                        | Description                                                                                                                                                                                                                                       |
|----------------------------|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>-e flag</b>             | <code>docker run -e NODE_ENV=prod nginx</code> | <ul style="list-style-type: none"> <li>Sets <b>single runtime variable</b> at container start</li> <li>visible in <code>docker inspect</code>, so unsuitable for secrets.</li> </ul>                                                              |
| <b>--env-file</b>          | <code>docker run --env-file .env nginx</code>  | <ul style="list-style-type: none"> <li>Loads variables from a <b>file</b>, one <code>VAR=value</code> per line</li> <li><code>#</code> starts a line comment; no shell-style <code>\$VAR</code> interpolation.</li> </ul>                         |
| <b>ENV in Dockerfile</b>   | <code>ENV NODE_ENV=production</code>           | <ul style="list-style-type: none"> <li>Bakes a variable into the <b>image</b> for build AND runtime</li> <li>persists in every container; bad for secrets (visible in <code>docker history</code>).</li> </ul>                                    |
| <b>ARG in Dockerfile</b>   | <code>ARG VERSION=1.0</code>                   | <ul style="list-style-type: none"> <li><b>Build-time only</b>, passed with <code>--build-arg</code></li> <li>not in the final image's runtime env, but still visible in <code>docker history</code>.</li> </ul>                                   |
| <b>Compose environment</b> | <code>environment:<br/>NODE_ENV: prod</code>   | <ul style="list-style-type: none"> <li>Sets variables on a service via <b>list or map syntax</b></li> <li>supports <code>\${VAR}</code> interpolation from host shell or project <code>.env</code> file.</li> </ul>                               |
| <b>Compose env_file</b>    | <code>env_file: webapp.env</code>              | <ul style="list-style-type: none"> <li>Loads container env from one or more <b>external files</b></li> <li>later files override earlier ones; distinct from the project <code>.env</code> used for <code>\${VAR}</code> interpolation.</li> </ul> |

TABLE 26

## Inspect and Debug

Once a container is running you need ways to look inside it, watch what the daemon is doing, see where your disk went, and get a shell into images that don't have one. This table covers the broader Docker inspect family — the `--format` Go template idiom for scripting, the system-level views ( `events` , `df` , `info` , `version` ), and Docker Desktop's `debug` and `init` helpers.

| Command                   | Example                                                       | Description                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker inspect</b>     | <code>docker inspect --format '{{.State.Status}}' web</code>  | <ul style="list-style-type: none"> <li>Returns low-level JSON config for any Docker object</li> <li>pair with <code>--format</code> Go templates to extract one field (canonical scripting pattern).</li> </ul>                                                                                                                                                |
| <b>docker events</b>      | <code>docker events --since 1h --filter type=container</code> | <ul style="list-style-type: none"> <li>Continuous stream of <b>daemon lifecycle events</b> (create, start, die, kill, oom, etc.)</li> <li>bound it with <code>--since</code> / <code>--until</code> and <code>--filter</code> or it blocks forever.</li> </ul>                                                                                                 |
| <b>docker system df</b>   | <code>docker system df -v</code>                              | <ul style="list-style-type: none"> <li>Reports Docker-managed disk usage across images, containers, volumes, and build cache</li> <li><code>-v</code> adds a per-item reclaimable breakdown.</li> </ul>                                                                                                                                                        |
| <b>docker system info</b> | <code>docker system info</code>                               | <ul style="list-style-type: none"> <li>Shows <b>runtime configuration</b> — storage driver, cgroup driver, kernel version, containerd/runc versions, container counts</li> <li>aliased as <code>docker info</code> .</li> </ul>                                                                                                                                |
| <b>docker version</b>     | <code>docker version</code>                                   | Prints client and server <b>version numbers</b> , the Engine <b>API version</b> , and component versions (containerd, runc, docker-init). Different from <code>docker info</code> .                                                                                                                                                                            |
| <b>docker debug</b>       | <code>docker debug web</code>                                 | <ul style="list-style-type: none"> <li>Attaches a <b>debug shell with its own toolbox</b> (vim, http, curl, plus Nix packages, installable from nixos.org) to any container or image, even distroless/scratch with no shell</li> <li>free with any signed-in Docker account (Docker Desktop only); changes never touch the real image or container.</li> </ul> |
| <b>docker init</b>        | <code>docker init</code>                                      | <ul style="list-style-type: none"> <li>Scaffolds <b>Dockerfile</b>, <b>compose.yaml</b>, <b>.dockerignore</b>, <b>README.Docker.md</b> for a new project</li> <li>templates for Go, Node, Python, Rust, Java, PHP, ASP.NET Core.</li> </ul>                                                                                                                    |

TABLE 27

## Copy Operations

How to move files in and out of containers with `docker cp` — a one-shot snapshot copy (not a live sync like a bind mount) that works on both running and stopped containers, making it the right tool for post-mortem retrieval and surgical config injection.

| Command                              | Example                                                 | Description                                                                                                                                      |
|--------------------------------------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker cp (container to host)</b> | <code>docker cp web:/app/log.txt ./log.txt</code>       | <ul style="list-style-type: none"> <li>Copies file from <b>container to host</b></li> <li>works on running and stopped containers.</li> </ul>    |
| <b>docker cp (host to container)</b> | <code>docker cp ./config.yml web:/app/config.yml</code> | <ul style="list-style-type: none"> <li>Copies file from <b>host to container</b></li> <li>immediately available in running container.</li> </ul> |
| <b>Copy directories</b>              | <code>docker cp web:/app/logs ./logs</code>             | Recursively copies <b>entire directories</b> (no <code>-r</code> flag — recursion is the default).                                               |
| <b>Archive mode</b>                  | <code>docker cp -a web:/app ./backup</code>             | <ul style="list-style-type: none"> <li>Preserves <b>UID/GID</b> and permissions</li> <li>equivalent to <code>cp -a</code> .</li> </ul>           |

TABLE 28

## BuildKit Advanced Features

BuildKit unlocks the modern Docker build pipeline: persistent caches that survive between builds, secret and SSH credentials that never touch image layers, attestations that prove what shipped, and Bake for orchestrating multi-target build matrices.

| Feature                       | Example                                                                                                                                                       | Description                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cache mounts</b>           | <code>RUN --mount=type=cache,target=/root/.cache/pip<br/>pip install -r requirements.txt</code>                                                               | <ul style="list-style-type: none"> <li>Mounts a <b>persistent package-manager cache</b> for one <code>RUN</code> step</li> <li>data lives in the builder, not in image layers, and is reused across builds.</li> </ul>                                                                                                                                                                                                                 |
| <b>SSH mounts</b>             | <code>RUN --mount=type=ssh git clone<br/>git@github.com:repo<br/>(invoke with <code>docker build --ssh<br/>default</code> )</code>                            | <ul style="list-style-type: none"> <li>Forwards the host's <b>SSH agent socket</b> into a single <code>RUN</code> step</li> <li>lets builds clone private Git repos without a key entering the image.</li> </ul>                                                                                                                                                                                                                       |
| <b>Bind mounts</b>            | <code>RUN --mount=type=bind,target=. go<br/>build -o /app/hello</code>                                                                                        | <ul style="list-style-type: none"> <li>Mounts build-context files into the step <b>without COPY</b></li> <li>files are read-only by default and never enter image layers or the cache.</li> </ul>                                                                                                                                                                                                                                      |
| <b>External cache</b>         | <code>docker buildx build --cache-from<br/>type=registry,ref=user/app:buildc<br/>ache --cache-to type=registry,ref<br/>=user/app:buildcache,mode=max .</code> | <ul style="list-style-type: none"> <li>Pushes/pulls build cache to a <b>remote location</b> (registry, S3, GHA)</li> <li>lets ephemeral CI runners share cache and stay fast.</li> </ul>                                                                                                                                                                                                                                               |
| <b>Inline cache</b>           | <code>docker buildx build --cache-to<br/>type=inline --cache-from &lt;image&gt;<br/>-t app .</code>                                                           | <ul style="list-style-type: none"> <li>Embeds cache metadata <b>inside the published image</b> itself</li> <li>simplest registry-backed cache, but only supports <code>mode=min</code> .</li> </ul>                                                                                                                                                                                                                                    |
| <b>SBOM attestation</b>       | <code>docker buildx build --sbom=true -t<br/>myapp .</code>                                                                                                   | <ul style="list-style-type: none"> <li>Attaches an <b>SPDX Software Bill of Materials</b> as an OCI attestation</li> <li>lists packages found in the image so scanners (Scout, Trivy) can flag CVEs</li> <li><b>BuildKit 0.32+</b> writes attestation manifests in the OCI artifact format by default, with an <code>oci-artifact</code> exporter option to fall back to the older layout for registries that lack support.</li> </ul> |
| <b>Provenance attestation</b> | <code>docker buildx build<br/>--provenance=mode=max -t myapp .</code>                                                                                         | <ul style="list-style-type: none"> <li>Attaches a <b>SLSA-format provenance</b> record proving what source, frontend, and parameters produced the image</li> <li><code>mode=max</code> adds the Dockerfile and LLB; <code>mode=min</code> is the default.</li> </ul>                                                                                                                                                                   |
| <b>Buildx bake</b>            | <code>docker buildx bake -f<br/>docker-bake.hcl<br/>(HCL defines <code>target</code> and <code>group</code> blocks)</code>                                    | <ul style="list-style-type: none"> <li>Runs multiple build targets <b>concurrently</b> from an HCL/JSON/Compose file</li> <li>supports variables, inheritance, and matrix targets for monorepos and multi-arch fan-out.</li> </ul>                                                                                                                                                                                                     |

TABLE 29

## Docker Scout Commands

Docker Scout is Docker's built-in supply-chain security tool: it reads an image's SBOM (generating one if missing), looks up known CVEs against it, and suggests safer base images. These CLI commands are the day-to-day surface — triaging vulnerabilities, comparing image versions, and enrolling an organization for policy enforcement.

| Command                             | Example                                                        | Description                                                                                                                                                                                          |
|-------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker scout cves</b>            | <code>docker scout cves<br/>nginx:latest</code>                | <ul style="list-style-type: none"> <li>Scans an image for <b>CVEs</b></li> <li>shows severity, CVSS score, affected package, and fix availability.</li> </ul>                                        |
| <b>docker scout quickview</b>       | <code>docker scout quickview<br/>nginx:latest</code>           | <ul style="list-style-type: none"> <li>Displays a <b>summary</b> of vulnerabilities and policy compliance</li> <li>fast overview without printing the full CVE list.</li> </ul>                      |
| <b>docker scout recommendations</b> | <code>docker scout<br/>recommendations<br/>nginx:latest</code> | <ul style="list-style-type: none"> <li>Suggests <b>base image</b> refresh or update</li> <li>shows benefits like fewer CVEs and smaller image size for each alternative.</li> </ul>                  |
| <b>docker scout sbom</b>            | <code>docker scout sbom<br/>nginx:latest</code>                | <ul style="list-style-type: none"> <li>Generates a <b>Software Bill of Materials</b></li> <li>lists every package and version (SPDX or CycloneDX output adds licenses and relationships).</li> </ul> |

| Command                     | Example                                                        | Description                                                                                                                                                                                                        |
|-----------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker scout compare</b> | <code>docker scout compare nginx:1.25 --to nginx:latest</code> | <ul style="list-style-type: none"> <li>• <b>Compares vulnerability profiles</b> of two image versions</li> <li>• canonical way to verify a base image bump actually reduced CVEs.</li> </ul>                       |
| <b>docker scout enroll</b>  | <code>docker scout enroll myorg</code>                         | <ul style="list-style-type: none"> <li>• <b>Enrolls an organization</b> in Docker Scout</li> <li>• required before policy enforcement, repository integration, and team dashboards work for org images.</li> </ul> |

TABLE 30

## Docker Model Runner

Docker Model Runner (DMR) is a Docker Desktop / Docker Engine feature for pulling, running, and serving local AI models with a `docker model` CLI that mirrors familiar image commands. Models are stored as OCI artifacts separately from container images, run on a host-native inference server (llama.cpp by default), and are reachable over OpenAI-, Anthropic-, and Ollama-compatible HTTP APIs. As of Docker Desktop 4.71, DMR is **disabled by default** and must be explicitly enabled in Settings.

| Command                          | Example                                                                           | Description                                                                                                                                                                                                                                                                                                                                               |
|----------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker model pull</b>         | <code>docker model pull ai/llama3.2</code>                                        | <ul style="list-style-type: none"> <li>• Downloads a model from <b>Docker Hub, an OCI registry, or Hugging Face</b></li> <li>• stored locally as OCI artifacts in a separate model store (not in <code>docker images</code> ).</li> </ul>                                                                                                                 |
| <b>docker model run</b>          | <code>docker model run ai/llama3.2</code>                                         | <ul style="list-style-type: none"> <li>• Runs a one-time prompt or starts an <b>interactive chat</b> in the terminal</li> <li>• the model is loaded on demand and unloaded after ~5 min idle.</li> </ul>                                                                                                                                                  |
| <b>docker model run --detach</b> | <code>docker model run -d ai/llama3.2</code>                                      | <ul style="list-style-type: none"> <li>• <b>Pre-loads</b> the model into memory without an interactive session</li> <li>• the OpenAI-compatible REST API is always served by Model Runner — <code>-d</code> just warms the cache.</li> </ul>                                                                                                              |
| <b>docker model list</b>         | <code>docker model list</code>                                                    | Lists locally pulled models with size, parameters, and quantization (alias: <code>docker model ls</code> ).                                                                                                                                                                                                                                               |
| <b>docker model rm</b>           | <code>docker model rm ai/llama3.2</code>                                          | Removes a model from the local model store to reclaim disk space.                                                                                                                                                                                                                                                                                         |
| <b>OpenAI-compatible API</b>     | <code>curl http://model-runner.docker.internal/engines/v1/chat/completions</code> | <ul style="list-style-type: none"> <li>• Reach the running model via <b>OpenAI-, Anthropic-, or Ollama-compatible endpoints</b> (including <code>/responses</code> for the OpenAI Responses API)</li> <li>• from containers use <code>model-runner.docker.internal</code> ; from the host use <code>http://localhost:12434/engines/v1</code> .</li> </ul> |

TABLE 31

## Docker Build Cloud and Docker Offload

Docker Build Cloud and Docker Offload both move Docker workloads off the laptop and onto managed cloud infrastructure, but at different scope. Build Cloud is a paid subscription add-on that runs `buildx` builds alone on a managed BuildKit instance in AWS (US East), with a shared persistent build cache and native `linux/amd64` + `linux/arm64` nodes; the CLI sends the build context to the remote builder and the resulting image streams back or is pushed to a registry, so it is a builder swap, same Dockerfile, same `docker buildx build` . Docker Offload, generally available since April 2026, goes further: it connects Docker Desktop to a full remote Docker daemon over a secure SSH tunnel, so `docker run` itself, not just `docker build` , executes in the cloud, and it is aimed at Docker Business subscribers and VDI environments that can't run local containers at all.

| Feature                         | Example                                                                     | Description                                                                                                                                                                                                                                                             |
|---------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Connect to cloud builder</b> | <code>docker buildx create --driver cloud myorg/mybuilder</code>            | <ul style="list-style-type: none"> <li>• <b>Registers a local endpoint</b> for an existing cloud builder (the builder itself is provisioned in the Docker Build Cloud Dashboard)</li> <li>• the local endpoint is named <code>cloud-myorg-mybuilder</code> .</li> </ul> |
| <b>Build with cloud</b>         | <code>docker buildx build --builder cloud-myorg-mybuilder -t myapp .</code> | <ul style="list-style-type: none"> <li>• Sends the build context to the remote builder</li> <li>• build runs on AWS EC2 with more CPU/RAM than the laptop, then result is loaded back or pushed to a registry.</li> </ul>                                               |

| Feature                             | Example                                                                                                               | Description                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker offload start</b>         | <code>docker offload start</code>                                                                                     | <ul style="list-style-type: none"> <li>Connects Docker Desktop to a secure remote Docker daemon over an <b>SSH tunnel</b>, so <code>docker run</code> and <code>docker build</code> both execute in the cloud</li> <li>requires a Docker Business subscription plus a Docker Offload subscription; port forwarding and bind mounts still behave like a local session.</li> </ul> |
| <b>Set as default builder</b>       | <code>docker buildx use cloud-myorg-mybuilder --global</code>                                                         | <ul style="list-style-type: none"> <li>Sets the cloud builder as the default for <code>docker buildx build</code></li> <li><code>--global</code> persists the selection across shells; affects <code>docker buildx build</code> only, not the legacy <code>docker build</code>.</li> </ul>                                                                                       |
| <b>List builders</b>                | <code>docker buildx ls</code>                                                                                         | <ul style="list-style-type: none"> <li>Lists all builder instances with their driver, endpoint, status, BuildKit version, and supported platforms</li> <li>the current default is marked with a <code>*</code>.</li> </ul>                                                                                                                                                       |
| <b>Shared build cache</b>           | (automatic; see <code>docker buildx du</code> )                                                                       | <ul style="list-style-type: none"> <li>Each cloud builder has a dedicated EBS volume holding a <b>persistent BuildKit cache shared by everyone using that builder</b></li> <li>the first teammate's build warms the cache; subsequent local and CI builds reuse it, so cold starts on fresh runners stop paying full rebuild cost.</li> </ul>                                    |
| <b>Native multi-platform builds</b> | <code>docker buildx build --builder cloud-myorg-mybuilder --platform linux/amd64,linux/arm64 -t myapp --push .</code> | <ul style="list-style-type: none"> <li>The cloud builder has <b>native</b> <code>linux/amd64</code> and <code>linux/arm64</code> nodes</li> <li>each platform runs on its real CPU in parallel with no QEMU emulation overhead, typically much faster than emulated cross-builds on a single-arch laptop or CI runner.</li> </ul>                                                |
| <b>docker offload stop / status</b> | <code>docker offload status</code><br><code>docker offload stop</code>                                                | <ul style="list-style-type: none"> <li><code>status</code> reports whether the session is idle, connecting, or running and its duration</li> <li><code>stop</code> tears the remote environment down immediately; an idle session auto-terminates after 5 minutes, so push images to a registry first to keep them.</li> </ul>                                                   |

TABLE 32

## Compose Dependency Conditions

The `depends_on` long syntax accepts a `condition` that controls *when* Compose considers a dependency satisfied before starting the dependent service. Picking the right condition is how you turn a fragile "container is up" hint into a real readiness gate — or a one-shot migration step.

| Condition                             | Example                                                                                                     | Description                                                                                                                                                                                                                                                             |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>service_started</b>                | <code>depends_on:</code><br><code>db:</code><br><code>condition: service_started</code>                     | <ul style="list-style-type: none"> <li>Default behavior — waits only for the dependency <b>container to be started</b></li> <li>does <b>not</b> wait for the app inside to be ready, so first connections can still fail.</li> </ul>                                    |
| <b>service_healthy</b>                | <code>depends_on:</code><br><code>db:</code><br><code>condition: service_healthy</code>                     | <ul style="list-style-type: none"> <li>Waits for the dependency's <code>healthcheck</code> <b>to report healthy</b></li> <li>requires a <code>HEALTHCHECK</code> in the image or a <code>healthcheck:</code> block; without one, the dependent never starts.</li> </ul> |
| <b>service_completed_successfully</b> | <code>depends_on:</code><br><code>migrate:</code><br><code>condition: service_completed_successfully</code> | <ul style="list-style-type: none"> <li>Waits for a one-off dependency to <b>exit with code 0</b></li> <li>canonical "run migrations first" pattern; non-zero exit blocks the dependent.</li> </ul>                                                                      |

TABLE 33

## Distroless and Minimal Images

Choosing a smaller base image shrinks the attack surface, the CVE footprint, and the pull/startup time — but each minimal base trades away something. This table compares the common minimal-base options so you can pick by what your app actually needs at runtime (a shell, a libc, a package manager, none of the above).

| Image                        | Example                                                 | Description                                                                                                                                                                                                                                                        |
|------------------------------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Alpine Linux                 | <code>FROM alpine:3.21</code>                           | <ul style="list-style-type: none"><li>• Lightweight <b>~5 MB base</b></li><li>• uses <b>musl libc</b> (not glibc), so prebuilt Python/Node binary wheels and other glibc-linked binaries may fail or need recompiling.</li></ul>                                   |
| Distroless                   | <code>FROM gcr.io/distroless/static-debian12</code>     | <ul style="list-style-type: none"><li>• <b>No shell, no package manager</b> — only your app and its runtime deps</li><li>• <code>static</code> is ~2 MB, ~50% of alpine's size; reduces attack surface and CVE noise.</li></ul>                                    |
| Scratch                      | <code>FROM scratch</code>                               | <ul style="list-style-type: none"><li>• Completely <b>empty base</b> — no libc, no shell, no files</li><li>• works only for <b>statically linked binaries</b> (e.g. Go with <code>CGO_ENABLED=0</code> , Rust with a musl target).</li></ul>                       |
| Distroless :debug variant    | <code>FROM gcr.io/distroless/base-debian12:debug</code> | <ul style="list-style-type: none"><li>• Adds a <b>busybox shell</b> on top of distroless so you can <code>docker run --entrypoint=sh</code></li><li>• intended for local debugging, never production.</li></ul>                                                    |
| Docker Hardened Images (DHI) | <code>FROM dhi.io/python:3.13</code>                    | <ul style="list-style-type: none"><li>• Docker's <b>distroless-based, near-zero-CVE</b> images on alpine or debian foundations</li><li>• free under Apache 2.0 since Dec 17 2025, ship with SBOM + SLSA provenance attestations, patched on a tight SLA.</li></ul> |

TABLE 34

## Docker AI (Gordon)

Gordon is Docker's AI-powered assistant available from Docker Desktop 4.74+ and the `docker ai` CLI. It reads your live Docker environment — running containers, logs, images, Compose files — proposes actions, and executes them only after your approval. Gordon on Docker Hub and docs.docker.com is free with no Docker account required; CLI and Docker Desktop usage counts against a usage plan.

| Command                           | Example                                                          | Description                                                                                                                                                                                                                                 |
|-----------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>docker ai</code> (TUI)      | <code>docker ai</code>                                           | <ul style="list-style-type: none"><li>• Opens an <b>interactive terminal UI</b> for Gordon</li><li>• type questions or tasks; Gordon proposes actions and waits for <code>y</code> approval before executing.</li></ul>                     |
| <code>docker ai</code> (one-shot) | <code>docker ai "show me logs from my nginx container"</code>    | <ul style="list-style-type: none"><li>• Runs a <b>one-off prompt</b> without the full TUI</li><li>• Gordon reads the environment, proposes relevant commands (e.g. <code>docker logs</code> ), and runs them on approval.</li></ul>         |
| Dockerfile review                 | <code>docker ai "review my Dockerfile for best practices"</code> | <ul style="list-style-type: none"><li>• Analyzes the <b>Dockerfile in the current directory</b> and suggests optimizations: smaller base, layer order, cache hits, secret handling</li><li>• Gordon can apply the edits in-place.</li></ul> |
| Debug container failures          | <code>docker ai "why did my db container crash"</code>           | <ul style="list-style-type: none"><li>• Reads container <b>logs and inspect output</b>, diagnoses the root cause, and suggests a fix</li><li>• any proposed <code>docker</code> commands shown before execution.</li></ul>                  |
| Gordon in Docker Desktop          | (Docker Desktop sidebar → Gordon)                                | <ul style="list-style-type: none"><li>• Same capabilities as CLI Gordon but with a <b>graphical session sidebar</b></li><li>• lets you link a project directory and run queries with richer visual output.</li></ul>                        |

TABLE 35

## Docker MCP Toolkit

The Docker MCP Toolkit ( `docker mcp` ) makes it trivial to install, run, and connect 120+ containerized Model Context Protocol (MCP) servers — GitHub, Playwright, Slack, Grafana, and more — to AI clients such as Claude Desktop, Cursor, VS Code, and Gordon. Each server runs in an isolated container with Docker's security model. Requires Docker Desktop 4.62+.

| Command                                 | Example                                                                                                      | Description                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker mcp profile create</b>        | <code>docker mcp profile create --name web-dev</code>                                                        | <ul style="list-style-type: none"> <li>Creates a <b>named profile</b> — a workspace that organizes a set of MCP servers</li> <li>profiles are the unit of sharing; push/pull via <code>docker mcp profile push/pull &lt;oci-ref&gt;</code> .</li> </ul>                                                                        |
| <b>docker mcp profile server add</b>    | <code>docker mcp profile server add web-dev --server catalog://mcp/docker-mcp-catalog/github-official</code> | <ul style="list-style-type: none"> <li>Adds a server to a profile using a <b>catalog URI</b> ( <code>catalog://</code> ), Docker image URI ( <code>docker://</code> ), local YAML ( <code>file://</code> ), or HTTPS registry URL</li> <li>servers requiring OAuth show "Configuration Required" in Docker Desktop.</li> </ul> |
| <b>docker mcp gateway run</b>           | <code>docker mcp gateway run --profile web-dev</code>                                                        | <ul style="list-style-type: none"> <li>Starts the <b>MCP Gateway</b> over <code>stdio</code> with the given profile's servers</li> <li>used to connect any AI client manually via its MCP JSON config ( <code>"command": "docker", "args": ["mcp", "gateway", "run", "--profile", "web-dev"]</code> ).</li> </ul>              |
| <b>docker mcp client connect</b>        | <code>docker mcp client connect vscode --profile my-project</code>                                           | <ul style="list-style-type: none"> <li>Connects a <b>named supported client</b> (claude, cursor, vscode, etc.) to a profile</li> <li>writes the appropriate client-specific config file (e.g. <code>.vscode/mcp.json</code> ).</li> </ul>                                                                                      |
| <b>docker mcp catalog server ls</b>     | <code>docker mcp catalog server ls mcp/docker-mcp-catalog</code>                                             | <ul style="list-style-type: none"> <li>Lists all servers in the <b>Docker MCP Catalog</b> (or a custom catalog) by name and ID</li> <li>the Server ID is used in <code>catalog://mcp/docker-mcp-catalog/&lt;server-id&gt;</code> URIs.</li> </ul>                                                                              |
| <b>docker mcp profile list</b>          | <code>docker mcp profile list</code>                                                                         | <ul style="list-style-type: none"> <li>Lists all local profiles</li> <li>use <code>docker mcp profile show &lt;id&gt;</code> for servers and config in one profile</li> </ul>                                                                                                                                                  |
| <b>docker mcp profile export/import</b> | <code>docker mcp profile export web-dev .docker/mcp-profile.json</code>                                      | <ul style="list-style-type: none"> <li>Exports a profile to a JSON file for <b>version-control sharing</b> with teammates</li> <li>credentials are excluded; recipients configure secrets separately after <code>docker mcp profile import</code> .</li> </ul>                                                                 |

TABLE 36

## Docker DHI CLI

The `docker dhi` CLI plugin (Docker Desktop 4.65+) manages Docker Hardened Images: browse the catalog, view SBOM/provenance attestations, mirror images into your organization's registry, and create customized variants that add your own packages on top of a DHI base. A standalone `dhictl` binary is also available for headless CI environments.

| Command                            | Example                                                                        | Description                                                                                                                                                                                                                                              |
|------------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker dhi catalog list</b>     | <code>docker dhi catalog list --filter golang</code>                           | <ul style="list-style-type: none"> <li>Lists available DHI images; filter by name, <code>--fips</code> , or <code>--stig</code></li> <li><code>docker dhi catalog get golang</code> shows available tags and CVE counts for a specific image.</li> </ul> |
| <b>docker dhi attestation sbom</b> | <code>docker dhi attestation sbom dhi/nginx:1.27</code>                        | <ul style="list-style-type: none"> <li>Displays the <b>SPDX SBOM</b> attached to a DHI image as an OCI attestation</li> <li><code>docker dhi attestation list dhi/nginx:1.27</code> shows all attestation types including provenance.</li> </ul>         |
| <b>docker dhi mirror start</b>     | <code>docker dhi mirror start --org my-org dhi/golang,my-org/dhi-golang</code> | <ul style="list-style-type: none"> <li>Begins <b>automatically mirroring</b> a DHI image into your Docker Hub organization registry</li> <li><code>--dependencies</code> also mirrors base layers; Docker patches the mirror on its SLA.</li> </ul>      |
| <b>docker dhi mirror list</b>      | <code>docker dhi mirror list --org my-org</code>                               | Lists all DHI images currently mirrored to your organization with their status.                                                                                                                                                                          |

| Command                                 | Example                                                                                                                         | Description                                                                                                                                                                                                                                                                      |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>docker dhi customization prepare</b> | <pre>docker dhi customization prepare golang 1.25 --org my-org --destination my-org/dhi-golang &gt; my-customization.yaml</pre> | <ul style="list-style-type: none"> <li>Generates a <b>YAML scaffold</b> for creating a custom variant of a DHI image</li> <li>edit the YAML to add packages or configuration, then <code>docker dhi customization create</code> .</li> </ul>                                     |
| <b>docker dhi customization create</b>  | <pre>docker dhi customization create my-customization.yaml --org my-org</pre>                                                   | <ul style="list-style-type: none"> <li>Submits the customization YAML to Docker to <b>build a custom DHI variant</b> and push it to your org registry</li> <li><code>docker dhi customization build logs &lt;id&gt; &lt;build-id&gt;</code> streams the build output.</li> </ul> |
| <b>docker dhi auth apk/deb</b>          | <pre>docker dhi auth apk</pre>                                                                                                  | <ul style="list-style-type: none"> <li>Generates <b>enterprise package repository credentials</b> for installing Docker-patched packages in your own images</li> <li><code>apk</code> for Alpine-based images, <code>deb</code> for Debian-based images.</li> </ul>              |

## References

### Official Documentation

- Docker Documentation — Home
- Dockerfile Reference
- Docker Engine Overview
- docker build Reference
- docker ps Reference
- docker logs Reference
- docker network Reference
- docker image Reference
- docker tag Reference
- docker compose up Reference
- docker compose build Reference
- Compose Watch (Develop) Documentation
- Compose `healthcheck` Reference
- Compose `networks` Reference
- Compose `secrets` Reference
- Compose `profiles` Reference
- Compose `include` Reference
- Docker Build Multi-stage Builds
- BuildKit Overview
- docker buildx bake Reference
- Docker Networking Overview
- Overlay Networks
- Macvlan Networks
- Docker Volumes
- tmpfs Mounts
- Swarm Services
- Docker Registry Overview
- Docker Hub Automated Builds
- Docker Security Overview
- Seccomp Security Profiles
- Docker Daemon Configuration ( `daemon.json` )
- Docker Remote API / SDK
- docker scout CLI Reference
- Docker Model Runner Overview
- docker model Reference
- Docker Model Runner — Supported Models
- Docker Build Cloud Usage
- Docker Build Cloud — Managing Cache
- Docker AI Tools Overview
- Docker MCP Toolkit — Get Started
- Docker MCP Catalog
- Docker CLI Reference
- Docker Compose File Reference
- Docker Desktop Overview
- docker run Reference
- docker exec Reference
- docker inspect Reference
- docker volume Reference
- docker push / docker pull
- docker compose Reference
- docker compose down Reference
- docker compose watch Reference
- Compose `depends_on` Reference
- Compose `deploy` (Swarm/resources) Reference
- Compose `volumes` Reference
- Compose `configs` Reference
- Compose `models` Element Reference
- Compose GPU Support
- Docker Build Cache
- docker buildx Overview
- Build Secrets
- Bridge Networks
- Host Networking
- IPvlan Networks
- Bind Mounts
- Docker Swarm Overview
- Swarm Secrets
- Docker Hub Documentation
- Content Trust / Notary
- Docker Rootless Mode
- AppArmor Security Profiles
- Docker Context Overview
- Docker Extension SDK
- Docker Scout Overview
- Docker Model Runner API Reference
- docker model pull Reference
- Docker Build Cloud Setup
- Docker Build Cloud — Multi-platform Builds
- Gordon (docker ai) Overview
- Docker MCP Toolkit Overview
- Docker MCP Toolkit CLI Reference
- Docker Hardened Images Overview

83. Docker DHI CLI Reference
85. Docker DHI — Mirror Images
87. Docker Engine 29 Release Notes
89. Docker Desktop Release Notes (4.x)
91. Docker Compose Migration (v1 → v2)
93. json-file Logging Driver
95. syslog Logging Driver
97. fluentd Logging Driver
99. gelf Logging Driver
101. Docker Healthcheck
103. Docker ARG Instruction
105. Docker COPY --chown / --chmod
107. Docker RUN --mount
109. Multi-platform Builds
111. HEALTHCHECK Instruction
113. docker system df Reference
115. docker events Reference
117. docker diff Reference
119. docker export / import Reference
121. docker commit Reference
123. docker manifest Reference
125. Docker socket protection
127. No-new-privileges flag
129. Docker Official Images on Docker Hub
131. Alpine Linux Docker Image
133. Docker Compose — Develop (Watch) Spec
135. Compose `restart: true` in `depends_on`
137. Compose `healthcheck.start_interval`
139. gvisor-tap-vsock (rootless network driver)
141. Docker Engine 29.5.0 Release Notes
143. OpenAI Responses API
145. Docker Offload — About
147. Compose `pre_start` / `post_start` / `pre_stop` Lifecycle Hooks Reference
149. docker debug Reference
84. Docker DHI — Browse Catalog
86. Docker DHI — Customization
88. Docker Engine 28 Release Notes
90. Docker Compose v2 Release Notes
92. Logging Drivers
94. local Logging Driver
96. journald Logging Driver
98. awslogs Logging Driver
100. loki Logging Driver (Grafana)
102. Docker .dockerignore Reference
104. Docker ENV Instruction
106. Docker COPY --link
108. Docker Attestations (SBOM / Provenance)
110. Docker BuildKit Environment Variables
112. docker stats Reference
114. docker system prune Reference
116. docker top Reference
118. docker cp Reference
120. docker save / load Reference
122. docker history Reference
124. docker trust Reference
126. User Namespaces
128. Read-only Containers
130. Google Distroless Images — GitHub
132. Scratch Base Image
134. Compose `initial_sync` action
136. Compose `required: false` in `depends_on`
138. CVE-2025-62725 — Compose include OCI path traversal
140. docker ps `.HealthStatus` format placeholder
142. OpenAI Chat Completions API (Docker Model Runner endpoint)
144. Compose v2 → v5 Version History
146. Docker Offload — Quickstart
148. BuildKit v0.32.0 Release Notes (GitHub)
150. Deprecated and Retired Docker Products and Features

## Technical Blogs & Tutorials

151. Docker Blog — Official Announcements
153. Docker Blog — Gordon GA (Docker AI)
155. Docker Blog — Docker Model Runner
157. Basil Teo — Docker Compose Evolution: v1 to v5 (2025)
159. Docker Security Cheat Sheet (OWASP)
161. Earthly Blog — Docker Multistage Build Best Practices (2025)
163. Ivan Velichko — Container Networking Deep Dive
165. Thoughtworks Technology Radar — Docker
167. Hadolint — Dockerfile Linter (GitHub)
169. Gripe — Vulnerability Scanner (Anchore)
171. Docker Bench for Security (GitHub)
173. Dockle — Container Image Linter (GitHub)
175. Chainguard Images (hardened minimal images)
177. Podman vs Docker (Red Hat overview)
179. BuildKit Docs — Secrets in builds
181. Docker Compose — GPU Reservation Spec
183. Docker Compose — Using Secrets
185. Docker Contexts — Managing Multiple Docker Endpoints
187. Docker Engine on Linux — Post-install Steps
189. Docker Desktop — Dev Environments (deprecated)
152. Docker Blog — Hardened Images Go Free (Dec 2025)
154. Docker Blog — MCP Toolkit
156. Ajeet Raina — Docker MCP CLI Cheat Sheet (2025)
158. Dive Tool — Exploring Docker Image Layers (GitHub)
160. Docker Best Practices for Dockerfile (official)
162. Bret Fisher — Docker Mastery Course Notes
164. Nigel Poulton — Docker Deep Dive (2025 edition)
166. Slim.AI — Container Minification
168. Trivy — Vulnerability Scanner for Containers (GitHub)
170. Syft — SBOM Generator for Containers (Anchore)
172. CIS Docker Benchmark
174. Container Structure Tests (Google)
176. Wolflinux — Chainguard's glibc-based undistro
178. nerdctl — containerd CLI with Docker-compatible syntax (GitHub)
180. Docker Compose Watch — File Sync Deep Dive (Docker Docs)
182. Docker Compose — Using Profiles
184. Docker Hub Rate Limiting Guide
186. Docker Engine on Linux — Installation (Ubuntu)
188. Docker Desktop for Mac — VirtioFS Performance
190. Docker Desktop — Extensions Marketplace
191. Docker / Docker CLI (GitHub)
192. Moby / Moby (Docker Engine — GitHub)

- 193. Docker / Compose (GitHub)
- 195. Docker / Docker Hub Tool (GitHub)
- 197. Docker / Model Runner (GitHub)
- 199. Docker / MCP Gateway (GitHub)
- 201. GoogleContainerTools / Distroless (GitHub)
- 203. Aquasecurity / Trivy (GitHub)
- 205. containers/gvisor-tap-vsock (GitHub)
- 207. OCI Runtime Spec (GitHub)
- 209. Runc — OCI Runtime Reference Implementation (GitHub)
- 211. Docker Samples (GitHub)
- 213. Docker Desktop DHI Demo App (GitHub)
- 194. Docker / Buildx (GitHub)
- 196. Docker / Scout CLI (GitHub)
- 198. Docker / Gordon AI (GitHub)
- 200. Docker / DHI CLI (GitHub)
- 202. Wagooodman / Dive — Docker Image Explorer (GitHub)
- 204. Anchore / Syft (GitHub)
- 206. OCI Image Spec (GitHub)
- 208. Containerd (GitHub)
- 210. awesome-docker (GitHub)
- 212. Dockerfile Best Practices Example Repo (GitHub)

## Video Resources

- 214. Docker — Official YouTube Channel
- 216. Fireship — 100 Seconds of Docker
- 218. Bret Fisher — Docker Compose v2 Deep Dive
- 220. Traversy Media — Docker Crash Course
- 215. TechWorld with Nana — Docker Tutorial for Beginners (2M+ views)
- 217. NetworkChuck — Docker Networking (600K+ views)
- 219. KodeKloud — Docker for Absolute Beginners
- 221. Docker DockerCon 2025 Keynote

## Industry Best Practice Guides & Books

- 222. Docker: Up & Running, 3rd Edition (O'Reilly)
- 224. The Docker Book (James Turnbull)
- 226. NIST SP 800-190 — Application Container Security Guide
- 228. Red Hat — Container Security Best Practices
- 230. 12-Factor App Methodology
- 232. Sigstore / Cosign — Container Signing
- 223. Docker Deep Dive, Nigel Poulton (2025 Edition)
- 225. Container Security (O'Reilly — Liz Rice)
- 227. Snyk — Docker Security Best Practices (2025)
- 229. Google Cloud — Containers Best Practices
- 231. OpenSSF — Securing the Software Supply Chain (GitHub Actions / Docker)